Centro Universitário do Distrito Federal – UDF

Disciplina: Modelagem de Banco de Dados

Professora: MsC Josyane Lannes Florenzano de Souza

PROJETO FINAL DE BANCO DE DADOS RELACIONAL

Sistema de Delivery de Restaurante

Modelagem conceitual, lógica e física e consultas SQL

Integrantes do grupo

Bernardo Kopp Pinheiro

Gabriel Valois Rodrigues

Pedro Henrique Lima dos Santos Mota

Piêtro Bitencourt Nunes

Zamorano Fragoso de Sousa

Brasília – DF, novembro de 2026 · Semestre 2026/2

Sumário

1. Modelagem Conceitual . . . 3

1.1 Descrição do cenário proposto . . . 3

1.2 Modelo Entidade-Relacionamento (MER) . . . 3

2. Modelagem Lógica . . . 7

2.1 Diagrama Entidade-Relacionamento (DER) . . . 7

2.2 Etapas da iteração na modelagem (do MER ao DER) . . . 8

3. Modelagem Física . . . 11

3.1 Ambiente e ordem de execução . . . 11

3.2 Glossário dos comandos e cláusulas usados no script . . . 12

3.3 Script DDL explicado . . . 12

3.4 Execução do script de criação . . . 17

3.5 Carga de dados . . . 18

4. Consultas SQL . . . 19

4.1 Consultas básicas . . . 19

4.2 Consultas intermediárias . . . 22

4.3 Consultas avançadas: Triggers e Stored Procedures . . . 34

5. Considerações Finais . . . 41

Requisitos atendidos . . . 41

Arquivos do projeto . . . 41

Referências . . . 41

Apêndice A – Script 01_schema_delivery.sql (completo) . . . 42

Apêndice B – Script 02_dados_delivery.sql (completo) . . . 47

Apêndice C – Script 03_consultas_delivery.sql (completo) . . . 52

1. Modelagem Conceitual

1.1 Descrição do cenário proposto

O cenário escolhido é um **sistema de delivery de restaurantes**: uma plataforma que conecta **clientes**, **restaurantes parceiros** e **entregadores**. O banco de dados precisa registrar toda a operação, ou seja, quem faz o pedido, de qual restaurante, o que foi pedido, onde deve ser entregue, quem faz a entrega, como o pedido foi pago e como o cliente avaliou o serviço.

O cliente se cadastra informando nome, CPF, telefone e e-mail e pode ter vários endereços (casa, trabalho), sendo um deles o principal. Cada restaurante possui um endereço e mantém um cardápio de produtos, que são organizados em categorias (pizzas, hambúrgueres, bebidas, sobremesas etc.). Ao fazer um pedido, o cliente escolhe um restaurante, os produtos e as quantidades, e informa o endereço de entrega. O pedido passa por diferentes situações (pendente, preparando, saiu para entrega, entregue ou cancelado). Quando sai para entrega, um entregador é associado a ele. Cada pedido tem um pagamento, feito por uma forma de pagamento (Pix, cartão, dinheiro...), e, depois de entregue, pode receber uma avaliação com nota e comentário.

Regras de negócio

1. Um cliente pode ter um ou mais endereços cadastrados, e um endereço pode ser compartilhado por mais de um cliente. O relacionamento guarda um apelido (ex.: "Casa") e se o endereço é o principal.
2. Todo endereço pertence a exatamente uma cidade. Endereços são reutilizados por clientes, restaurantes e entregas.
3. Um restaurante tem exatamente um endereço e oferece um ou mais produtos. Cada produto pertence a um único restaurante e a uma única categoria.
4. O produto tem preço, estoque e um indicador de ativo (produtos inativos não podem ser vendidos).
5. Um pedido é feito por um único cliente a um único restaurante e é entregue em um endereço.
6. Um pedido contém um ou mais produtos; para cada um, registram-se a quantidade e o preço unitário praticado no momento da compra.
7. O entregador (nome, CPF, telefone, tipo de veículo, placa e situação) só é associado ao pedido quando ele sai para entrega; por isso a relação é opcional.
8. Cada pedido possui exatamente um pagamento, com valor, data, situação e uma forma de pagamento.
9. Um pedido entregue pode receber, no máximo, uma avaliação (nota de 1 a 5 e comentário).
10. Ao vender, o estoque do produto diminui; se o pedido for cancelado, o estoque é devolvido. O valor total do pedido é a soma dos itens mais a taxa de entrega.

Informações que o sistema deve responder

O modelo foi pensado para responder perguntas de negócio como: quais produtos mais vendem; quanto cada restaurante faturou; qual a nota média de cada restaurante; quais clientes nunca pediram; quantas entregas cada entregador concluiu; quais pedidos estão em andamento; e qual o estoque disponível. Essas perguntas guiam as consultas da seção 4.

1.2 Modelo Entidade-Relacionamento (MER)

O MER foi construído na ferramenta **BRModelo**. A notação usada é a seguinte:

- **Retângulo**: entidade (objeto sobre o qual guardamos informações).
- **Círculo vazio**: atributo; **círculo preenchido**: identificador (chave primária).

- **Losango**: relacionamento entre entidades. Relacionamentos N:N podem ter atributos próprios (ex.: *contém* tem quantidade e preco_unitario).
- **Cardinalidade (mín,máx)** no padrão de **leitura cruzada** (*look across*): o par escrito ao lado de uma entidade indica quantas ocorrências dela se relacionam com uma ocorrência da outra entidade.

> **Exemplo de leitura:** em *CLIENTE (1,1) — realiza — PEDIDO (0,N)*, o (0,N) ao lado de PEDIDO significa que um cliente realiza de 0 a N pedidos; e o (1,1) ao lado de CLIENTE significa que um pedido é realizado por exatamente 1 cliente.

O MER completo está na figura da próxima página (página inteira): **11 entidades** e **13 relacionamentos**, dos quais 2 são N:N (*possui* entre CLIENTE e ENDERECO, e *contém* entre PEDIDO e PRODUTO).

Entidades e atributos

Entidade	Descrição	Atributos (identificador em negrito)
CLIENTE	Pessoa que faz pedidos.	**id_cliente**, nome, cpf, telefone, email, data_cadastro
ENDERECO	Local físico reutilizável por clientes, restaurantes e entregas.	**id_endereco**, logradouro, numero, complemento, bairro, cep
CIDADE	Cidade e unidade federativa do endereço.	**id_cidade**, nome, uf
RESTAURANTE	Parceiro que prepara os pedidos.	**id_restaurante**, nome_fantasia, cnpj, telefone
CATEGORIA	Agrupa produtos (Pizzas, Bebidas...).	**id_categoria**, nome
PRODUTO	Item do cardápio de um restaurante.	**id_produto**, nome, descricao, preco, estoque, ativo
ENTREGADOR	Profissional que leva o pedido ao cliente.	**id_entregador**, nome, cpf, telefone, tipo_veiculo, placa, ativo
PEDIDO	Compra feita por um cliente em um restaurante.	**id_pedido**, data_hora, status, taxa_entrega, valor_total
FORMA_PAGAMENTO	Meios de pagamento aceitos.	**id_forma_pagamento**, descricao
PAGAMENTO	Cobrança do pedido.	**id_pagamento**, valor, data_pagamento, status
AVALIACAO	Opinião do cliente sobre o pedido entregue.	**id_avaliacao**, nota, comentario, data_avaliacao

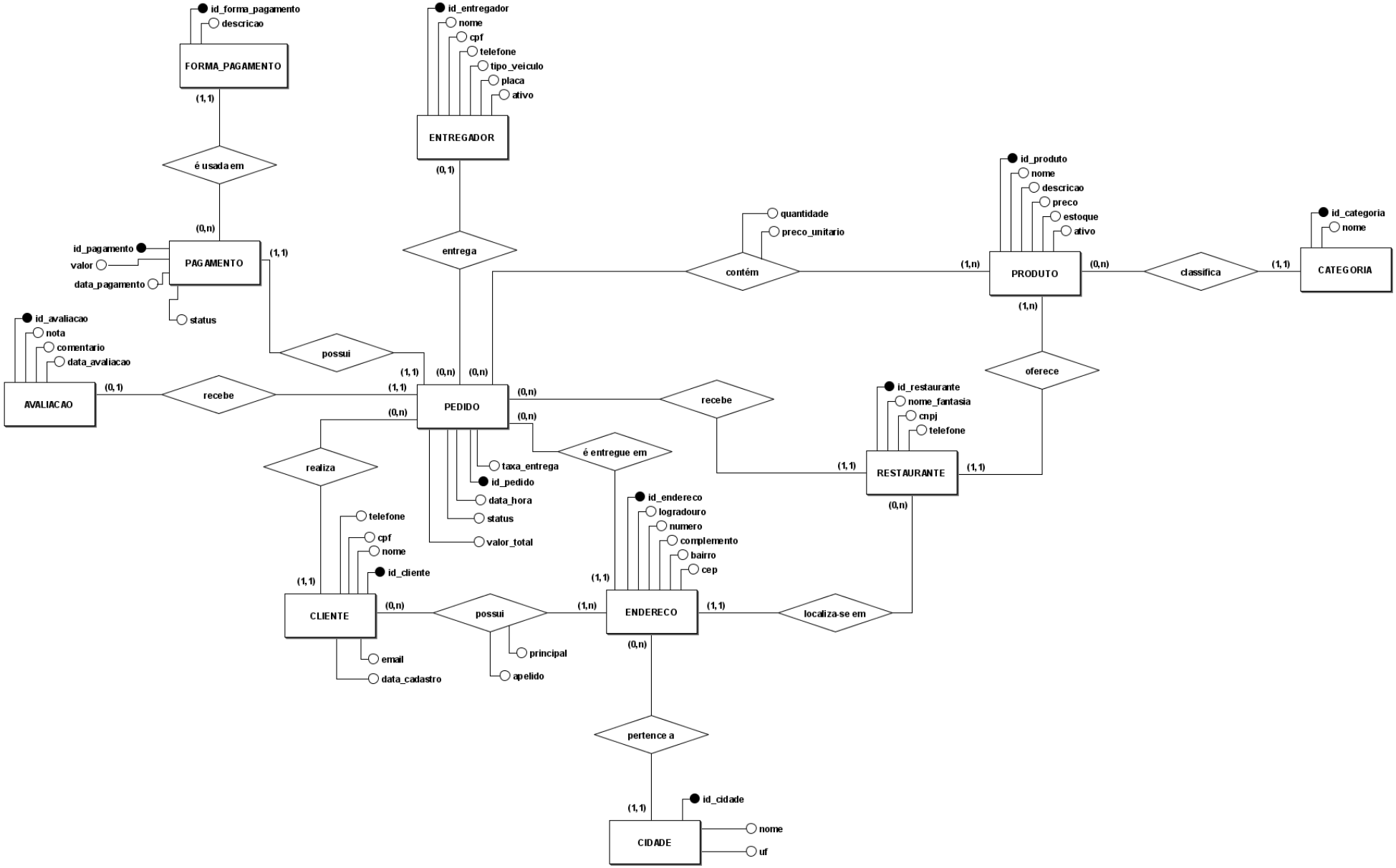


Figura 1 – Modelo Entidade-Relacionamento (BRModelo), cardinalidades em leitura cruzada

Relacionamentos e cardinalidades (leitura cruzada)

#	Relacionamento	Cardinalidades	Leitura
1	possui (CLIENTE–ENDERECO)	CLIENTE (0,N) ENDERECO (1,N)	Um cliente possui de 1 a N endereços; um endereço pode ser de 0 a N clientes. Atributos: apelido, principal. **(N:N)**
2	localiza-se em (RESTAURANTE–ENDERECO)	RESTAURANTE (0,N) ENDERECO (1,1)	Um restaurante localiza-se em exatamente 1 endereço; um endereço pode abrigar de 0 a N restaurantes.
3	pertence a (ENDERECO–CIDADE)	ENDERECO (0,N) CIDADE (1,1)	Um endereço pertence a exatamente 1 cidade; uma cidade tem de 0 a N endereços.
4	realiza (CLIENTE–PEDIDO)	CLIENTE (1,1) PEDIDO (0,N)	Um cliente realiza de 0 a N pedidos; um pedido é realizado por exatamente 1 cliente.
5	recebe (RESTAURANTE–PEDIDO)	RESTAURANTE (1,1) PEDIDO (0,N)	Um restaurante recebe de 0 a N pedidos; um pedido é feito a exatamente 1 restaurante.
6	entrega (ENTREGADOR–PEDIDO)	ENTREGADOR (0,1) PEDIDO (0,N)	Um entregador entrega de 0 a N pedidos; um pedido tem 0 ou 1 entregador (só recebe um ao sair para entrega).
7	é entregue em (PEDIDO–ENDERECO)	PEDIDO (0,N) ENDERECO (1,1)	Um pedido é entregue em exatamente 1 endereço; um endereço recebe de 0 a N entregas.
8	oferece (RESTAURANTE–PRODUTO)	RESTAURANTE (1,1) PRODUTO (1,N)	Um restaurante oferece de 1 a N produtos; um produto é oferecido por exatamente 1 restaurante.
9	classifica (CATEGORIA–PRODUTO)	CATEGORIA (1,1) PRODUTO (0,N)	Uma categoria classifica de 0 a N produtos; um produto pertence a exatamente 1 categoria.
10	contém (PEDIDO–PRODUTO)	PEDIDO (0,N) PRODUTO (1,N)	Um pedido contém de 1 a N produtos; um produto aparece em 0 a N pedidos. Atributos: quantidade, preco_unitario. **(N:N)**
11	possui (PEDIDO–PAGAMENTO)	PEDIDO (1,1) PAGAMENTO (1,1)	Um pedido possui exatamente 1 pagamento e um pagamento pertence a exatamente 1 pedido. **(1:1)**
12	é usada em (FORMA_PAGAMENTO–PAGAMENTO)	FORMA_PAGAMENTO (1,1) PAGAMENTO (0,N)	Uma forma de pagamento é usada em 0 a N pagamentos; um pagamento usa exatamente 1 forma.
13	recebe (PEDIDO–AVALIACAO)	PEDIDO (1,1) AVALIACAO (0,1)	Um pedido recebe 0 ou 1 avaliação; uma avaliação refere-se a exatamente 1 pedido. **(1:1 opcional)**

2. Modelagem Lógica

2.1 Diagrama Entidade-Relacionamento (DER)

O DER foi construído no **MySQL Workbench (Data Modeling)** e traduz o MER para o modelo relacional: **13 tabelas** (11 entidades + 2 tabelas associativas), com colunas, tipos de dados, chaves primárias e chaves estrangeiras. A notação do Workbench é a de "pé de galinha" (*crow's foot*):

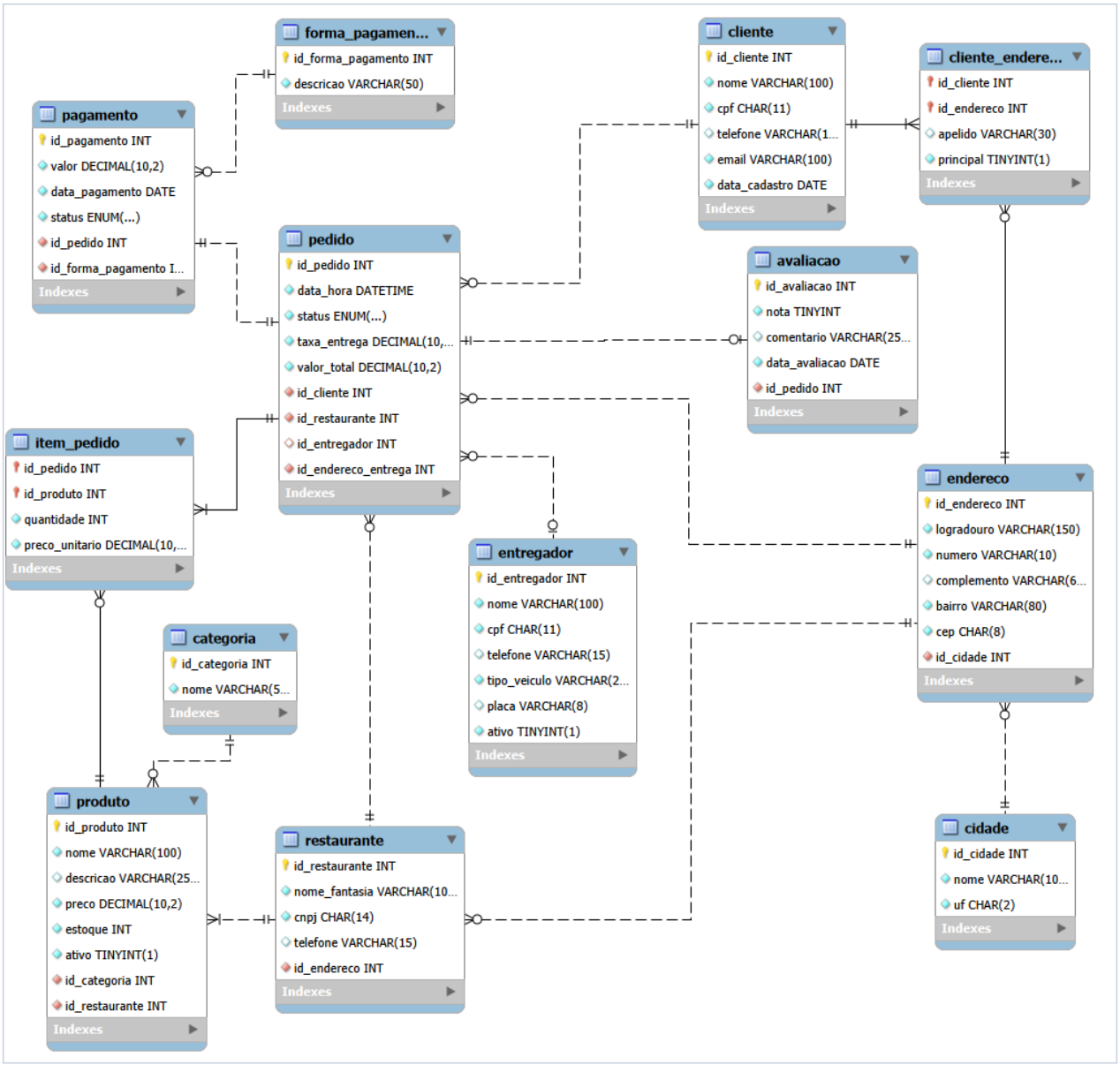


Figura 2 – Diagrama Entidade-Relacionamento (MySQL Workbench)

Símbolo	Significado
||	um e somente um (obrigatório)
o|	zero ou um (opcional)
|<	um ou muitos
o<	zero ou muitos
linha contínua / tracejada	relacionamento identificador (a FK faz parte da PK) / não identificador

Tabelas do modelo lógico

Tabela	Chave primária	Chaves estrangeiras	Origem no MER
cidade	id_cidade	—	Entidade CIDADE
endereco	id_endereco	id_cidade → cidade	Entidade ENDERECO + *pertence a*
cliente	id_cliente	—	Entidade CLIENTE
cliente_endereco	(id_cliente, id_endereco)	id_cliente → cliente id_endereco → endereco	Relacionamento N:N *possui*
restaurante	id_restaurante	id_endereco → endereco	Entidade RESTAURANTE + *localiza-se em*
categoria	id_categoria	—	Entidade CATEGORIA
produto	id_produto	id_categoria → categoria id_restaurante → restaurante	Entidade PRODUTO + *classifica* e *oferece*
entregador	id_entregador	—	Entidade ENTREGADOR
pedido	id_pedido	id_cliente, id_restaurante, id_entregador (opcional), id_endereco_entrega	Entidade PEDIDO + *realiza*, *recebe*, *entrega*, *é entregue em*
item_pedido	(id_pedido, id_produto)	id_pedido → pedido id_produto → produto	Relacionamento N:N *contém*
forma_pagamento	id_forma_pagamento	—	Entidade FORMA_PAGAMENTO
pagamento	id_pagamento	id_pedido (único), id_forma_pagamento	Entidade PAGAMENTO + *possui* e *é usada em*
avaliacao	id_avaliacao	id_pedido (único)	Entidade AVALIACAO + *recebe*

2.2 Etapas da iteração na modelagem (do MER ao DER)

A passagem do modelo conceitual para o lógico foi feita em etapas sucessivas. Em cada uma aplicamos uma regra de mapeamento e conferimos o resultado contra o MER antes de seguir para a etapa seguinte.

Etapa 1 – Entidades viram tabelas

Cada uma das 11 entidades do MER virou uma tabela. Adotamos nomes em minúsculas, no singular, sem acentos e com underscore (*snake_case*), como *forma_pagamento*, para evitar problemas de acentuação e de diferença entre maiúsculas e minúsculas.

Etapa 2 – Atributos viram colunas com tipos de dados

Cada atributo virou uma coluna, e o tipo foi escolhido pelo que o dado representa:

Tipo	Onde foi usado	Motivo da escolha
INT	ids, estoque, quantidade	Números inteiros; ids gerados automaticamente.
VARCHAR(n)	nome, e-mail, logradouro, descrição, telefone	Texto de tamanho variável, até n caracteres.
CHAR(n)	cpf (11), cnpj (14), cep (8), uf (2)	Texto de tamanho fixo, guardado sem máscara.
DECIMAL(10,2)	preco, taxa_entrega, valor_total, valor	Número exato com 2 casas decimais, o tipo correto para dinheiro (FLOAT pode arredondar errado).
DATE / DATETIME	data_cadastro, data_pagamento / data_hora do pedido	DATE guarda só a data; DATETIME guarda data e hora.
TINYINT(1)	ativo, principal	Valor lógico: 0 = não, 1 = sim.
TINYINT	nota	Inteiro pequeno, suficiente para notas de 1 a 5.

Tipo	Onde foi usado	Motivo da escolha
ENUM	status do pedido e do pagamento	Lista fechada de valores; impede status escritos errado.

Etapa 3 – Identificadores viram chaves primárias

Os identificadores do MER (círculos preenchidos) viraram **chaves primárias** do tipo *chave substituta*: um número *id_* gerado automaticamente (*AUTO_INCREMENT*). Dados naturalmente únicos, como CPF, CNPJ e e-mail, continuam como chaves candidatas protegidas por *UNIQUE*.

Etapa 4 – Relacionamentos 1:N viram chaves estrangeiras

Nos relacionamentos 1:N, a **chave estrangeira (FK)** fica na tabela do lado "muitos", apontando para a chave primária do lado "um". Foram dez relacionamentos desse tipo, por exemplo: *pedido.id_cliente → cliente*, *produto.id_categoria → categoria* e *endereco.id_cidade → cidade*. Cada FK tem um nome (*fk_tabela_referencia*) para facilitar a identificação de erros.

Etapa 5 – Relacionamentos N:N viram tabelas associativas

Um relacionamento N:N não pode ser representado diretamente no modelo relacional, então ele vira uma tabela própria com duas FKs que, juntas, formam a **chave primária composta**:

- **contém** (PEDIDO–PRODUTO) → **item_pedido** (id_pedido, id_produto), com *quantidade* e *preco_unitario*. O preço unitário é copiado no momento da compra para que alterações futuras no preço do produto não mudem pedidos antigos.
- **possui** (CLIENTE–ENDERECO) → **cliente_endereco** (id_cliente, id_endereco), com *apelido* e *principal*.

Como essas tabelas dependem das duas tabelas de origem, os relacionamentos são *identificadores* (linha contínua no DER).

Etapa 6 – Relacionamentos 1:1

Em *possui* (PEDIDO–PAGAMENTO) e *recebe* (PEDIDO–AVALIACAO) a cardinalidade é 1:1. A FK *id_pedido* foi colocada nas tabelas **pagamento** e **avaliacao**, pois elas só existem a partir de um pedido, e recebeu a restrição **UNIQUE**, que impede dois pagamentos (ou duas avaliações) para o mesmo pedido.

Etapa 7 – Cardinalidade mínima (obrigatório × opcional)

- **Mínimo 1 do lado "um"** (ex.: o pedido tem exatamente 1 cliente): a FK é *NOT NULL*.
- **Mínimo 0** (ex.: o pedido pode não ter entregador, ENTREGADOR (0,1)): a FK *id_entregador* aceita *NULL*.
- **Mínimo 1 do lado "muitos"** (ex.: o pedido tem pelo menos 1 item): o banco não consegue garantir isso só com FK; a regra fica documentada no diagrama (símbolo |<) e é tratada na aplicação, apoiada pela procedure *sp_adicionar_item* (seção 4.3).

Etapa 8 – Normalização

- **1ª Forma Normal:** todos os atributos são atômicos. O endereço foi dividido em logradouro, número, bairro e CEP, e a cidade foi para uma tabela própria; não há campos com listas de valores.
- **2ª Forma Normal:** nas tabelas com chave composta, cada atributo depende da chave inteira: *quantidade* e *preco_unitario* dependem de (pedido, produto); *apelido* e *principal* dependem de (cliente, endereço).
- **3ª Forma Normal:** não há dependências transitivas. Nome e UF dependem só da cidade (tabela *cidade*); o nome da categoria e a descrição da forma de pagamento ficam em tabelas próprias, e o nome do restaurante não é repetido nos produtos.

> **Redundâncias conscientes:** *pedido.valor_total* é um valor derivado (soma dos itens + taxa), mantido por um trigger para acelerar consultas e relatórios; e *pagamento.valor* registra o valor efetivamente cobrado ou estornado, que pode diferir do pedido em caso de recusa ou estorno.

Etapa 9 – Integridade referencial e regras de validação

- **ON DELETE CASCADE** em *cliente_endereco → cliente* e *item_pedido → pedido*: se o "pai" for excluído, os registros que dependem exclusivamente dele são excluídos junto.
- **Padrão RESTRICT** nas demais FKs: não é possível excluir, por exemplo, um produto já vendido ou uma cidade com endereços.
- **CHECK** (preço, estoque e valores não negativos; quantidade maior que zero; nota entre 1 e 5), **ENUM** (status) e **UNIQUE** (CPF, CNPJ, e-mail) garantem a qualidade dos dados.

Etapa 10 – Verificação MER × DER

Com o DER montado, conferimos as 13 cardinalidades do MER contra os símbolos do Workbench (obrigatório/opcional e um/muitos) e ajustamos as relações que divergiam, como os dois 1:1 e o entregador opcional no pedido.

Refinamentos feitos ao longo da iteração

- **Cliente e endereço:** a primeira versão previa um único endereço por cliente (1:N). Como no delivery o cliente usa mais de um endereço (casa, trabalho), o relacionamento passou a N:N, criando a tabela *cliente_endereco*.
- **Endereço compartilhado:** como o mesmo cadastro de endereço serve a clientes, restaurantes e entregas, o lado do cliente ficou (0,N): um endereço de restaurante não pertence a nenhum cliente.
- **Leitura cruzada:** padronizamos as cardinalidades em *look across* e revisamos todas as 13 relações.
- **Entregador opcional:** o pedido só recebe entregador depois de sair para entrega, então a relação ficou (0,1).
- **DER final:** corrigimos os símbolos de cardinalidade e reorganizamos as tabelas no diagrama para que nenhuma linha atravesse outra tabela.

3. Modelagem Física

3.1 Ambiente e ordem de execução

A modelagem física foi implementada em **MySQL 8**, usando o **MySQL Workbench (SQL Development)**. O projeto tem três scripts, que devem ser executados nesta ordem:

Ordem	Arquivo	Conteúdo
1	01_schema_delivery.sql	DDL: cria o banco *delivery_restaurante* e as 13 tabelas, com chaves, restrições e regras de validação.
2	02_dados_delivery.sql	DML: insere os dados de exemplo (fictícios) em todas as tabelas.
3	03_consultas_delivery.sql	DQL: 26 consultas (6 básicas, 15 intermediárias e 5 avançadas, com triggers e procedures).

Os três scripts completos, exatamente como foram executados, estão nos **Apêndices A, B e C** ao final deste documento.

DDL (*Data Definition Language*) define a estrutura (CREATE, DROP); **DML** (*Data Manipulation Language*) manipula os dados (INSERT, UPDATE, DELETE); **DQL** (*Data Query Language*) consulta os dados (SELECT).

3.2 Glossário dos comandos e cláusulas usados no script

Termo	O que significa	Onde foi usado
DROP DATABASE IF EXISTS	Apaga o banco se ele já existir, permitindo rodar o script várias vezes sempre do zero (apaga todos os dados).	Início do script
CREATE DATABASE ... utf8mb4 / utf8mb4_unicode_ci	Cria o banco. *utf8mb4* é o conjunto de caracteres (aceita acentos e emojis) e *unicode_ci* é a regra de comparação sem diferenciar maiúsculas de minúsculas (*ci* = case insensitive).	Início do script
USE	Seleciona o banco em que os próximos comandos serão executados.	Início do script
ENGINE=InnoDB	Motor de armazenamento da tabela. O InnoDB é o único que suporta **chave estrangeira**, **transações** (COMMIT/ROLLBACK) e bloqueio por linha; motores antigos como o MyISAM ignoram as FKs.	Todas as tabelas
AUTO_INCREMENT	O MySQL gera sozinho o próximo número da coluna (1, 2, 3...).	PKs simples (id_...)
NOT NULL / NULL	Coluna obrigatória / opcional (aceita ausência de valor).	Todas as colunas
DEFAULT	Valor usado quando o INSERT não informa a coluna.	status, ativo, estoque, principal, taxa_entrega
CURRENT_TIMESTAMP	Data e hora atuais do servidor (equivale a NOW()). Com DEFAULT, o pedido já nasce com a data e a hora do momento.	pedido.data_hora
PRIMARY KEY	Chave primária: identifica cada linha de forma única e não aceita NULL. Pode ser composta (duas ou mais colunas).	Todas; composta em item_pedido e cliente_endereco
UNIQUE KEY	Não permite dois valores iguais na coluna (ou na combinação de colunas).	cpf, cnpj, email, (nome, uf), id_pedido em pagamento e avaliacao
CONSTRAINT nome	Dá um nome a uma regra (FK, CHECK), facilitando identificar a regra violada numa mensagem de erro.	FKs e CHECKs
FOREIGN KEY ... REFERENCES	Chave estrangeira: só aceita valores que já existam na chave primária da tabela referenciada (integridade referencial).	15 relacionamentos
ON DELETE CASCADE	Ao excluir o registro "pai", exclui automaticamente os "filhos" que dependem dele.	cliente_endereco → cliente; item_pedido → pedido
RESTRICT (padrão)	Sem ON DELETE, o MySQL impede excluir um registro "pai" que ainda tenha "filhos".	Demais FKs
CHECK	Regra de validação avaliada a cada INSERT/UPDATE (exige MySQL 8.0.16 ou superior; em versões antigas é ignorada).	preco, estoque, valor, quantidade, nota
ENUM	A coluna só aceita um dos valores da lista.	pedido.status, pagamento.status

3.3 Script DDL explicado

Criação do banco de dados

```
DROP DATABASE IF EXISTS delivery_restaurante;
CREATE DATABASE delivery_restaurante
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;
USE delivery_restaurante;
```

O banco é recriado do zero a cada execução, com codificação que aceita acentos. As tabelas foram criadas na ordem em que as dependências exigem: primeiro as tabelas "pai" (cidade, cliente, categoria...) e depois as que referenciam outras por chave estrangeira.

Tabela cidade

Guarda cada cidade uma única vez. A restrição UNIQUE em (nome, uf) impede cidades duplicadas.

```
CREATE TABLE cidade (
id_cidade INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
uf CHAR(2) NOT NULL,
PRIMARY KEY (id_cidade),
UNIQUE KEY uq_cidade_nome_uf (nome, uf)
-- UNIQUE KEY composta: o par (nome, uf) não pode se repetir, ou seja,
-- não existem duas vezes "Goiânia/GO". Mas "Goiânia/GO" e "Goiânia/XX" seriam aceitas.
) ENGINE=InnoDB;
```

Tabela endereco

Endereço reutilizável por clientes, restaurantes e entregas. A FK id_cidade é obrigatória; o complemento é opcional (NULL).

```
CREATE TABLE endereco (
id_endereco INT NOT NULL AUTO_INCREMENT,
logradouro VARCHAR(150) NOT NULL,
numero VARCHAR(10) NOT NULL,    -- VARCHAR porque pode ter letras (ex.: "12A", "S/N")
complemento VARCHAR(60) NULL,   -- NULL = campo opcional
bairro VARCHAR(80) NOT NULL,
cep CHAR(8) NOT NULL,           -- CEP sem hífen: sempre 8 dígitos
id_cidade INT NOT NULL,          -- FK para cidade
PRIMARY KEY (id_endereco),
CONSTRAINT fk_endereco_cidade
FOREIGN KEY (id_cidade) REFERENCES cidade (id_cidade)
) ENGINE=InnoDB;
```

Tabela cliente

CPF e e-mail são UNIQUE: não existem dois clientes com o mesmo documento ou e-mail.

```
CREATE TABLE cliente (
id_cliente INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
cpf CHAR(11) NOT NULL,   -- CPF sem pontos e traço: 11 dígitos
telefone VARCHAR(15)  NULL,
email VARCHAR(100) NOT NULL,
-- DATE: guarda somente a data (AAAA-MM-DD).
data_cadastro DATE NOT NULL,
PRIMARY KEY (id_cliente),
-- UNIQUE: impede dois clientes com o mesmo CPF ou o mesmo e-mail.
UNIQUE KEY uq_cliente_cpf (cpf),
UNIQUE KEY uq_cliente_email (email)
) ENGINE=InnoDB;
```

Tabela cliente_endereco

Tabela associativa do N:N entre cliente e endereço. A PK composta impede vincular o mesmo endereço duas vezes ao mesmo cliente. O ON DELETE CASCADE remove os vínculos quando o cliente é excluído.

```
CREATE TABLE cliente_endereco (
id_cliente INT NOT NULL,
id_endereco INT NOT NULL,
apelido VARCHAR(30) NULL,
principal TINYINT(1) NOT NULL DEFAULT 0,   -- DEFAULT 0: se não informar, assume 0
-- PK COMPOSTA: a combinação (id_cliente, id_endereco) é única, então o
-- mesmo endereço não pode ser vinculado duas vezes ao mesmo cliente.
PRIMARY KEY (id_cliente, id_endereco),
CONSTRAINT fk_ce_cliente
FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)
ON DELETE CASCADE,
-- Para o endereço não usamos CASCADE: um endereço também pode estar em
-- uso por restaurantes ou pedidos, então não pode sumir sem cuidado.
CONSTRAINT fk_ce_endereco
FOREIGN KEY (id_endereco) REFERENCES endereco (id_endereco)
) ENGINE=InnoDB;
```

Tabela restaurante

CNPJ único e um endereço obrigatório.

```
CREATE TABLE restaurante (
id_restaurante INT NOT NULL AUTO_INCREMENT,
nome_fantasia VARCHAR(100) NOT NULL,
cnpj CHAR(14) NOT NULL,  -- CNPJ sem máscara: 14 dígitos
telefone VARCHAR(15)  NULL,
id_endereco INT NOT NULL,
PRIMARY KEY (id_restaurante),
UNIQUE KEY uq_restaurante_cnpj (cnpj),
CONSTRAINT fk_restaurante_endereco
FOREIGN KEY (id_endereco) REFERENCES endereco (id_endereco)
) ENGINE=InnoDB;
```

Tabela categoria

Categorias de produtos, com nome único.

```
CREATE TABLE categoria (
id_categoria INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(50) NOT NULL,
PRIMARY KEY (id_categoria),
UNIQUE KEY uq_categoria_nome (nome)
) ENGINE=InnoDB;
```

Tabela produto

Os CHECKs impedem preço ou estoque negativos. O campo ativo (0/1) permite tirar um item do cardápio sem apagá-lo, preservando o histórico de vendas.

```
CREATE TABLE produto (
id_produto INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
descricao VARCHAR(255) NULL,
preco DECIMAL(10,2) NOT NULL,
estoque INT NOT NULL DEFAULT 0,
ativo TINYINT(1) NOT NULL DEFAULT 1,  -- 1 = ativo no cardápio, 0 = inativo
id_categoria INT NOT NULL,
id_restaurante INT NOT NULL,
PRIMARY KEY (id_produto),
CONSTRAINT ck_produto_preco CHECK (preco >= 0),
CONSTRAINT ck_produto_estoque CHECK (estoque >= 0),
CONSTRAINT fk_produto_categoria
FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria),
CONSTRAINT fk_produto_restaurante
FOREIGN KEY (id_restaurante) REFERENCES restaurante (id_restaurante)
) ENGINE=InnoDB;
```

Tabela entregador

A placa é opcional, pois entregadores de bicicleta não têm placa. O campo ativo indica se o entregador está disponível.

```
CREATE TABLE entregador (
id_entregador INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
cpf CHAR(11) NOT NULL,
telefone VARCHAR(15) NULL,
tipo_veiculo VARCHAR(20) NOT NULL,       -- Moto, Bicicleta, Carro
placa VARCHAR(8) NULL,           -- NULL para quem entrega de bicicleta
ativo TINYINT(1) NOT NULL DEFAULT 1,
PRIMARY KEY (id_entregador),
UNIQUE KEY uq_entregador_cpf (cpf)
) ENGINE=InnoDB;
```

Tabela pedido

Tabela central. data_hora recebe CURRENT_TIMESTAMP por padrão, status é um ENUM que começa em PENDENTE, e id_entregador aceita NULL até o pedido sair para entrega. id_endereco_entrega guarda o endereço daquela entrega.

```
CREATE TABLE pedido (
id_pedido INT NOT NULL AUTO_INCREMENT,
data_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
status ENUM('PENDENTE','PREPARANDO','SAIU_PARA_ENTREGA','ENTREGUE','CANCELADO') NOT NULL DEFAULT
    'PENDENTE',
taxa_entrega DECIMAL(10,2) NOT NULL DEFAULT 0.00,
-- valor_total = soma dos itens + taxa de entrega. É um valor DERIVADO
-- guardado de propósito: o trigger da parte de consultas o atualiza.
valor_total DECIMAL(10,2) NOT NULL DEFAULT 0.00,
id_cliente INT NOT NULL,
id_restaurante INT NOT NULL,
-- id_entregador aceita NULL: o pedido só recebe entregador quando sai
-- para entrega (relacionamento opcional, cardinalidade (0,1)).
id_entregador INT NULL,
-- Endereço onde ESTE pedido será entregue (pode ser diferente do
-- endereço principal do cliente).
id_endereco_entrega INT NOT NULL,
PRIMARY KEY (id_pedido),
CONSTRAINT ck_pedido_taxa CHECK (taxa_entrega >= 0),
CONSTRAINT ck_pedido_total CHECK (valor_total >= 0),
CONSTRAINT fk_pedido_cliente
FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente),
CONSTRAINT fk_pedido_restaurante
FOREIGN KEY (id_restaurante) REFERENCES restaurante (id_restaurante),
CONSTRAINT fk_pedido_entregador
FOREIGN KEY (id_entregador) REFERENCES entregador (id_entregador),
CONSTRAINT fk_pedido_endereco
FOREIGN KEY (id_endereco_entrega) REFERENCES endereco (id_endereco)
) ENGINE=InnoDB;
```

Tabela item_pedido

Tabela associativa do N:N entre pedido e produto, com PK composta. preco_unitario guarda o preço no momento da compra. O ON DELETE CASCADE remove os itens quando o pedido é excluído.

```
CREATE TABLE item_pedido (
id_pedido INT NOT NULL,
id_produto INT NOT NULL,
quantidade INT NOT NULL,
preco_unitario DECIMAL(10,2) NOT NULL,
PRIMARY KEY (id_pedido, id_produto), -- PK composta: um produto aparece só uma vez por pedido
CONSTRAINT ck_item_quantidade CHECK (quantidade > 0),
CONSTRAINT ck_item_preco CHECK (preco_unitario >= 0),
-- ON DELETE CASCADE: ao excluir um pedido, os itens dele são excluídos
-- junto (item sem pedido não faz sentido).
CONSTRAINT fk_item_pedido
FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido)
ON DELETE CASCADE,
-- Sem CASCADE em produto: não se pode excluir um produto que já foi vendido.
CONSTRAINT fk_item_produto
FOREIGN KEY (id_produto) REFERENCES produto (id_produto)
) ENGINE=InnoDB;
```

Tabela forma_pagamento

Meios de pagamento, com descrição única.

```
CREATE TABLE forma_pagamento (
id_forma_pagamento INT NOT NULL AUTO_INCREMENT,
descricao VARCHAR(50) NOT NULL,
PRIMARY KEY (id_forma_pagamento),
UNIQUE KEY uq_forma_descricao (descricao)
) ENGINE=InnoDB;
```

Tabela pagamento

O UNIQUE em id_pedido garante o 1:1 (um pagamento por pedido). O status é um ENUM (PENDENTE, APROVADO, RECUSADO, ESTORNADO).

```
CREATE TABLE pagamento (
id_pagamento INT NOT NULL AUTO_INCREMENT,
valor DECIMAL(10,2) NOT NULL,
data_pagamento DATE NOT NULL,
status ENUM('PENDENTE','APROVADO','RECUSADO','ESTORNADO') NOT NULL DEFAULT 'PENDENTE',
id_pedido INT NOT NULL,
id_forma_pagamento INT NOT NULL,
PRIMARY KEY (id_pagamento),
UNIQUE KEY uq_pagamento_pedido (id_pedido),
CONSTRAINT ck_pagamento_valor CHECK (valor >= 0),
CONSTRAINT fk_pagamento_pedido
FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido),
CONSTRAINT fk_pagamento_forma
FOREIGN KEY (id_forma_pagamento) REFERENCES forma_pagamento (id_forma_pagamento)
) ENGINE=InnoDB;
```

Tabela avaliacao

O UNIQUE em id_pedido garante no máximo uma avaliação por pedido. O CHECK limita a nota de 1 a 5.

```
CREATE TABLE avaliacao (
id_avaliacao INT NOT NULL AUTO_INCREMENT,
-- TINYINT: inteiro pequeno (-128 a 127), suficiente para notas de 1 a 5.
nota TINYINT NOT NULL,
comentario VARCHAR(255) NULL,
data_avaliacao DATE NOT NULL,
id_pedido INT NOT NULL,
PRIMARY KEY (id_avaliacao),
UNIQUE KEY uq_avaliacao_pedido (id_pedido),
-- CHECK ... BETWEEN 1 AND 5: só aceita notas de 1 a 5.
CONSTRAINT ck_avaliacao_nota CHECK (nota BETWEEN 1 AND 5),
CONSTRAINT fk_avaliacao_pedido
FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido)
) ENGINE=InnoDB;
```

3.4 Execução do script de criação

O script foi executado no MySQL Workbench. A saída (*Action Output*) mostra o banco apagado e recriado e as 13 tabelas criadas com sucesso.

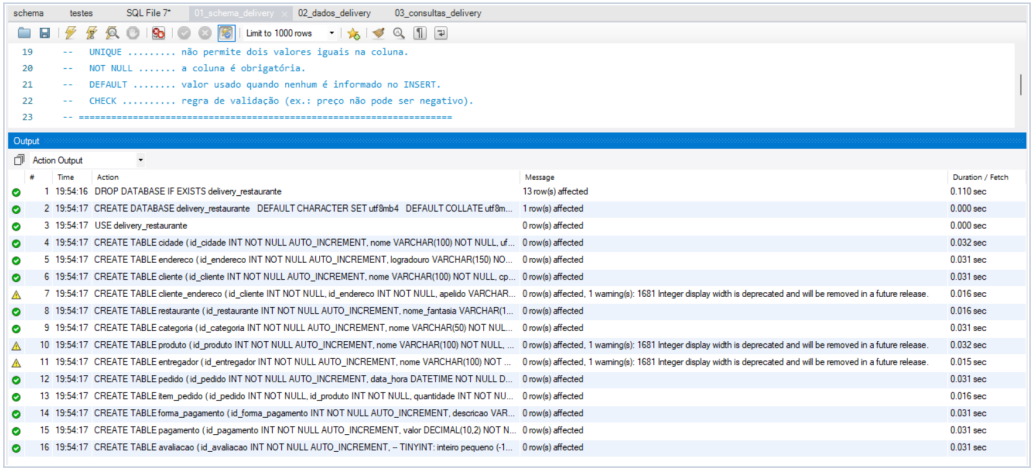


Figura 3 – Execução do script 01_schema_delivery.sql no MySQL Workbench

> **Sobre o aviso 1681 (*Integer display width is deprecated*):** aparece nas tabelas que usam TINYINT(1) (cliente_endereco, produto e entregador). É apenas um aviso de depreciação: nas versões recentes do MySQL a largura de exibição "(1)" será removida no futuro. Não impede a criação das tabelas nem altera o funcionamento do banco.

3.5 Carga de dados

O script 02 insere dados fictícios com INSERT, respeitando a ordem das dependências: cidade, endereco, cliente, cliente_endereco, restaurante, categoria, produto, entregador, forma_pagamento, pedido, item_pedido, pagamento e avaliacao. Os ids são informados manualmente para que as chaves estrangeiras fiquem fáceis de ler e conferir. O valor_total de cada pedido já está calculado (soma dos itens + taxa de entrega), por isso os triggers só devem ser criados depois da carga.

Tabela	Registros	Tabela	Registros
cidade	6	entregador	6
endereco	21	forma_pagamento	5
cliente	12	pedido	33
cliente_endereco	17	item_pedido	78
restaurante	4	pagamento	33
categoria	7	avaliacao	19
produto	24	**Total**	**265**

Foram incluídos **casos de borda de propósito**, para que as consultas tenham o que mostrar: dois clientes que nunca pediram; um entregador que nunca entregou e outro inativo; um produto inativo e sem estoque; pedidos cancelados, pendentes e em andamento (com id_entregador NULL); datas de agosto a novembro de 2026; e avaliações em apenas parte dos pedidos entregues.



Figura 4 – Execução do script 02_dados_delivery.sql: 13 INSERTs sem erros e sem avisos

4. Consultas SQL

Foram criadas **26 consultas**, acima do mínimo exigido de 20: **6 básicas**, **15 intermediárias** e **5 avançadas** (mínimo pedido: 4, 4 e 2). Todas foram executadas no MySQL Workbench sobre os dados do script 02, e os resultados abaixo são as capturas de tela dessas execuções.

Nível	Consultas	Conceitos
Básicas (4.1)	B1 a B6	SELECT, FROM, escolha de colunas, apelidos (AS)
Intermediárias (4.2)	I1 a I15	DISTINCT, colunas calculadas, WHERE com operadores lógicos, LIKE, BETWEEN, IN, funções de agregação, INNER JOIN, LEFT JOIN, GROUP BY, HAVING, ORDER BY/LIMIT, subconsultas e UNION
Avançadas (4.3)	A1 a A5	Triggers (BEFORE e AFTER) e Stored Procedures

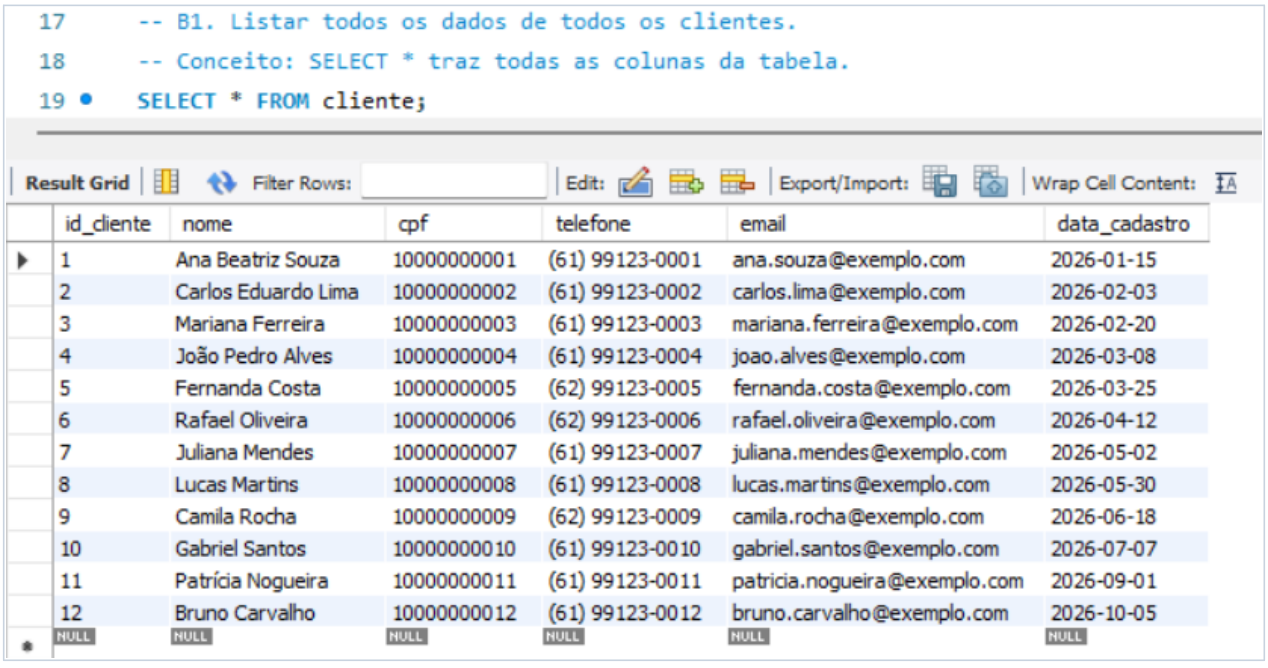
4.1 Consultas básicas

B1. Listar todos os dados de todos os clientes.

Conceito: SELECT * traz todas as colunas da tabela.

```
SELECT * FROM cliente;
```

Resultado: Os 12 clientes cadastrados, com todas as colunas da tabela.



```
17    -- B1. Listar todos os dados de todos os clientes.
18    -- Conceito: SELECT * traz todas as colunas da tabela.
19    SELECT * FROM cliente;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_cliente	nome	cpf	telefone	email	data_cadastro
1	Ana Beatriz Souza	10000000001	(61) 99123-0001	ana.souza@exemplo.com	2026-01-15
2	Carlos Eduardo Lima	10000000002	(61) 99123-0002	carlos.lima@exemplo.com	2026-02-03
3	Mariana Ferreira	10000000003	(61) 99123-0003	mariana.ferreira@exemplo.com	2026-02-20
4	João Pedro Alves	10000000004	(61) 99123-0004	joao.alves@exemplo.com	2026-03-08
5	Fernanda Costa	10000000005	(62) 99123-0005	fernanda.costa@exemplo.com	2026-03-25
6	Rafael Oliveira	10000000006	(62) 99123-0006	rafael.oliveira@exemplo.com	2026-04-12
7	Juliana Mendes	10000000007	(61) 99123-0007	juliana.mendes@exemplo.com	2026-05-02
8	Lucas Martins	10000000008	(61) 99123-0008	lucas.martins@exemplo.com	2026-05-30
9	Camila Rocha	10000000009	(62) 99123-0009	camila.rocha@exemplo.com	2026-06-18
10	Gabriel Santos	10000000010	(61) 99123-0010	gabriel.santos@exemplo.com	2026-07-07
11	Patrícia Nogueira	10000000011	(61) 99123-0011	patricia.nogueira@exemplo.com	2026-09-01
12	Bruno Carvalho	10000000012	(61) 99123-0012	bruno.carvalho@exemplo.com	2026-10-05
NULL	NULL	NULL	NULL	NULL	NULL

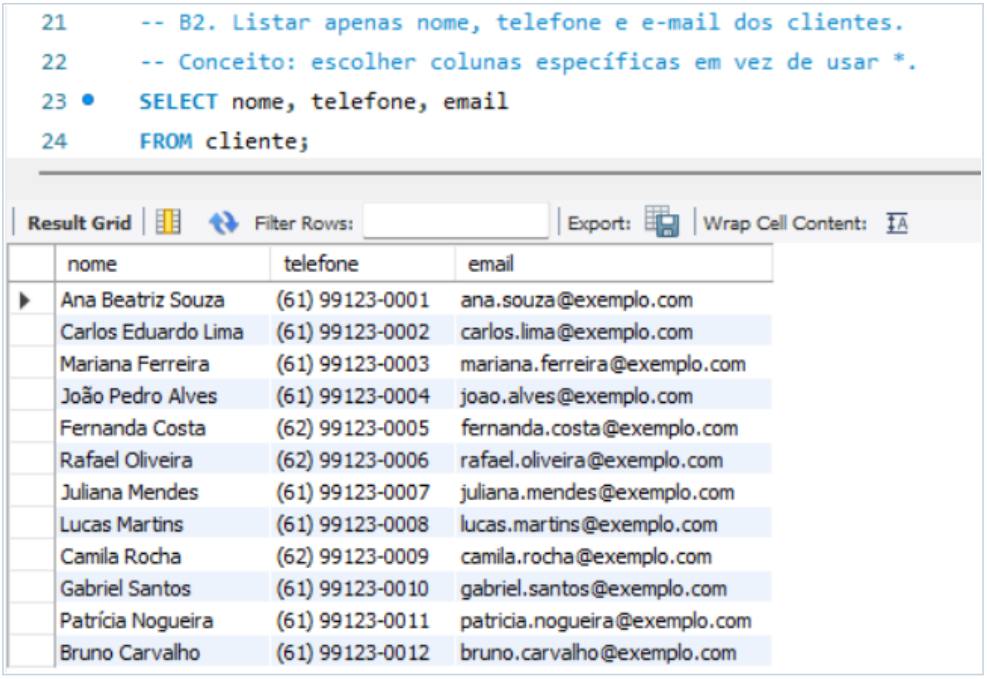
Figura 5 – Resultado da consulta B1 no MySQL Workbench

B2. Listar apenas nome, telefone e e-mail dos clientes.

Conceito: escolher colunas específicas em vez de usar *.

```
SELECT nome, telefone, email
FROM cliente;
```

Resultado: Os mesmos 12 clientes, agora só com nome, telefone e e-mail.



nome	telefone	email
Ana Beatriz Souza	(61) 99123-0001	ana.souza@exemplo.com
Carlos Eduardo Lima	(61) 99123-0002	carlos.lima@exemplo.com
Mariana Ferreira	(61) 99123-0003	mariana.ferreira@exemplo.com
João Pedro Alves	(61) 99123-0004	joao.alves@exemplo.com
Fernanda Costa	(62) 99123-0005	fernanda.costa@exemplo.com
Rafael Oliveira	(62) 99123-0006	rafael.oliveira@exemplo.com
Juliana Mendes	(61) 99123-0007	juliana.mendes@exemplo.com
Lucas Martins	(61) 99123-0008	lucas.martins@exemplo.com
Camila Rocha	(62) 99123-0009	camila.rocha@exemplo.com
Gabriel Santos	(61) 99123-0010	gabriel.santos@exemplo.com
Patrícia Nogueira	(61) 99123-0011	patricia.nogueira@exemplo.com
Bruno Carvalho	(61) 99123-0012	bruno.carvalho@exemplo.com

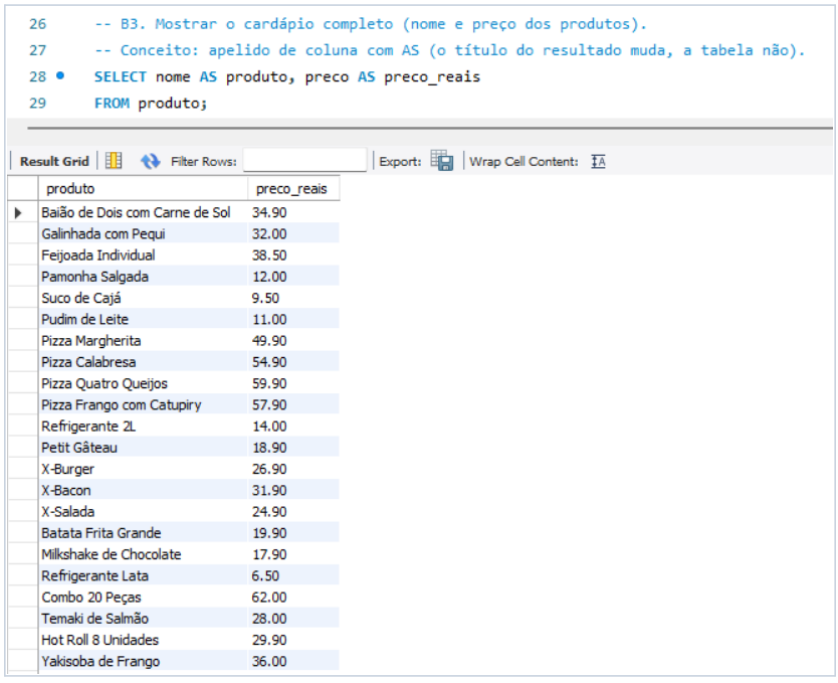
Figura 6 – Resultado da consulta B2 no MySQL Workbench

B3. Mostrar o cardápio completo (nome e preço dos produtos).

Conceito: apelido de coluna com AS (o título do resultado muda, a tabela não).

```
SELECT nome AS produto, preco AS preco_reais
FROM produto;
```

Resultado: O cardápio de 24 produtos, com as colunas renomeadas pelos apelidos (produto, preco_reais). O print mostra os primeiros itens.



produto	preco_reais
Baião de Dois com Carne de Sol	34.90
Galinhada com Pequi	32.00
Feijoada Individual	38.50
Pamonha Salgada	12.00
Suco de Cajá	9.50
Pudim de Leite	11.00
Pizza Margherita	49.90
Pizza Calabresa	54.90
Pizza Quatro Queijos	59.90
Pizza Frango com Catupiry	57.90
Refrigerante 2L	14.00
Petit Gâteau	18.90
X-Burger	26.90
X-Bacon	31.90
X-Salada	24.90
Batata Frita Grande	19.90
Milkshake de Chocolate	17.90
Refrigerante Lata	6.50
Combo 20 Peças	62.00
Temaki de Salmão	28.00
Hot Roll 8 Unidades	29.90
Yakisoba de Frango	36.00

Figura 7 – Resultado da consulta B3 no MySQL Workbench

B4. Listar os restaurantes cadastrados.

```
SELECT nome_fantasia, cnpj, telefone
FROM restaurante;
```

Resultado: Os 4 restaurantes parceiros.

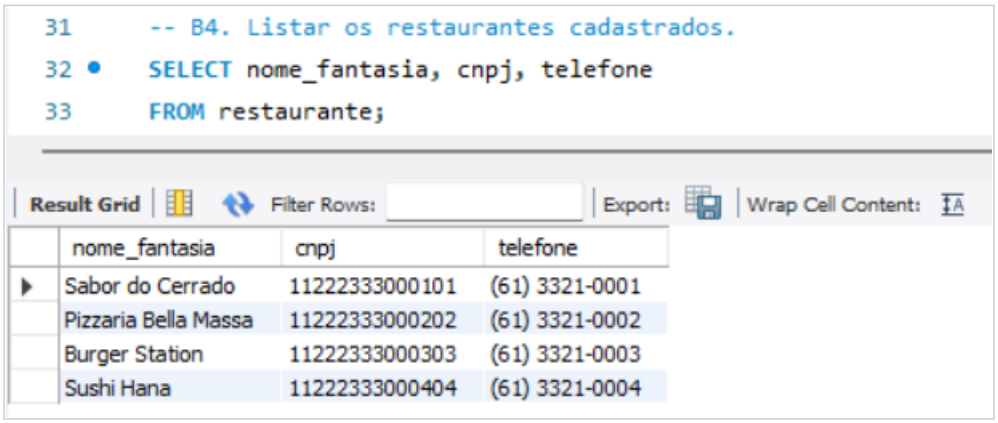


Figura 8 – Resultado da consulta B4 no MySQL Workbench

B5. Listar os entregadores e seus veículos.

```
SELECT nome AS entregador, tipo_veiculo, placa
FROM entregador;
```

Resultado: Os 6 entregadores. Os que usam bicicleta aparecem com NULL na placa (campo opcional).

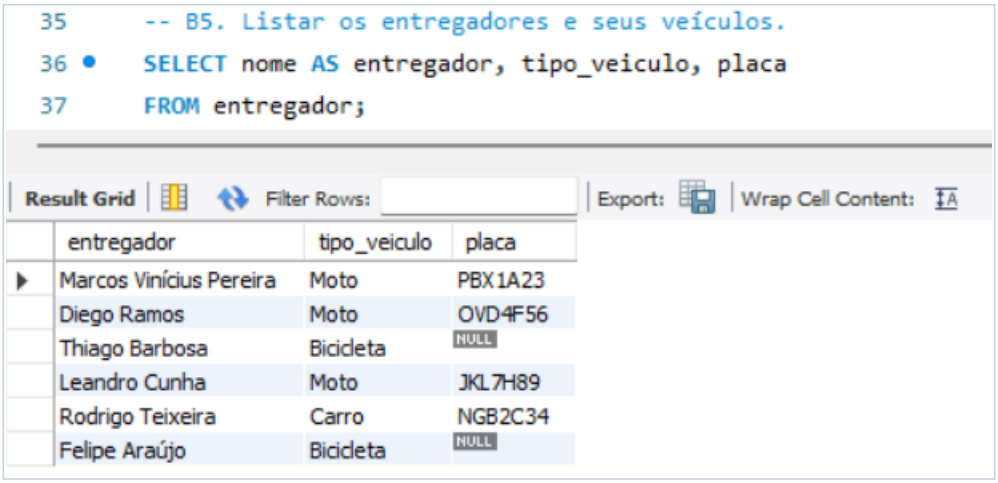
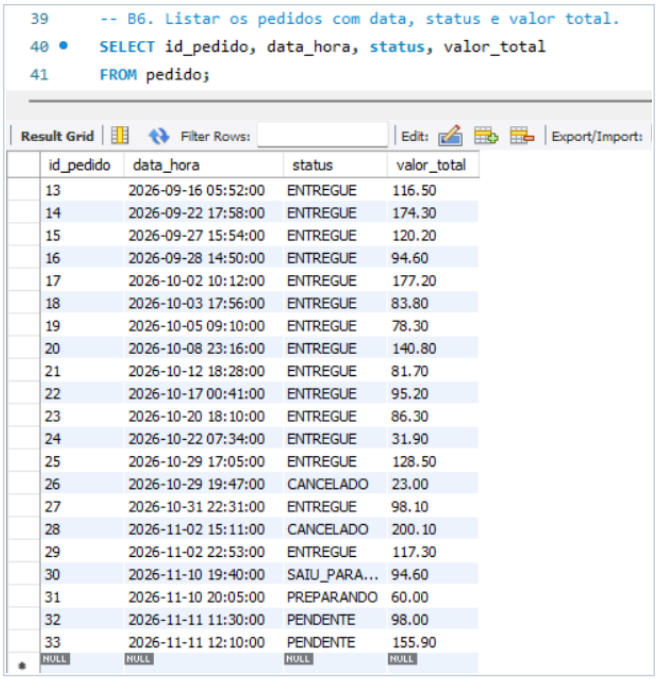


Figura 9 – Resultado da consulta B5 no MySQL Workbench

B6. Listar os pedidos com data, status e valor total.

```
SELECT id_pedido, data_hora, status, valor_total
FROM pedido;
```

Resultado: Pedidos com data, status e valor. O print mostra a lista a partir do pedido 13, o que exibe os diferentes status: ENTREGUE, CANCELADO, SAIU_PARA_ENTREGA, PREPARANDO e PENDENTE.



id_pedido	data_hora	status	valor_total
13	2026-09-16 05:52:00	ENTREGUE	116.50
14	2026-09-22 17:58:00	ENTREGUE	174.30
15	2026-09-27 15:54:00	ENTREGUE	120.20
16	2026-09-28 14:50:00	ENTREGUE	94.60
17	2026-10-02 10:12:00	ENTREGUE	177.20
18	2026-10-03 17:56:00	ENTREGUE	83.80
19	2026-10-05 09:10:00	ENTREGUE	78.30
20	2026-10-08 23:16:00	ENTREGUE	140.80
21	2026-10-12 18:28:00	ENTREGUE	81.70
22	2026-10-17 00:41:00	ENTREGUE	95.20
23	2026-10-20 18:10:00	ENTREGUE	86.30
24	2026-10-22 07:34:00	ENTREGUE	31.90
25	2026-10-29 17:05:00	ENTREGUE	128.50
26	2026-10-29 19:47:00	CANCELADO	23.00
27	2026-10-31 22:31:00	ENTREGUE	98.10
28	2026-11-02 15:11:00	CANCELADO	200.10
29	2026-11-02 22:53:00	ENTREGUE	117.30
30	2026-11-10 19:40:00	SAIU_PARA...	94.60
31	2026-11-10 20:05:00	PREPARANDO	60.00
32	2026-11-11 11:30:00	PENDENTE	98.00
33	2026-11-11 12:10:00	PENDENTE	155.90
NULL	NULL	NULL	NULL

Figura 10 – Resultado da consulta B6 no MySQL Workbench

4.2 Consultas intermediárias

I1. Quais tipos de veículo os entregadores usam (sem repetição)?

Conceito: DISTINCT remove linhas repetidas do resultado.

```
SELECT DISTINCT tipo_veiculo
FROM entregador;
```

Resultado: Existem três tipos de veículo: Moto, Bicicleta e Carro.

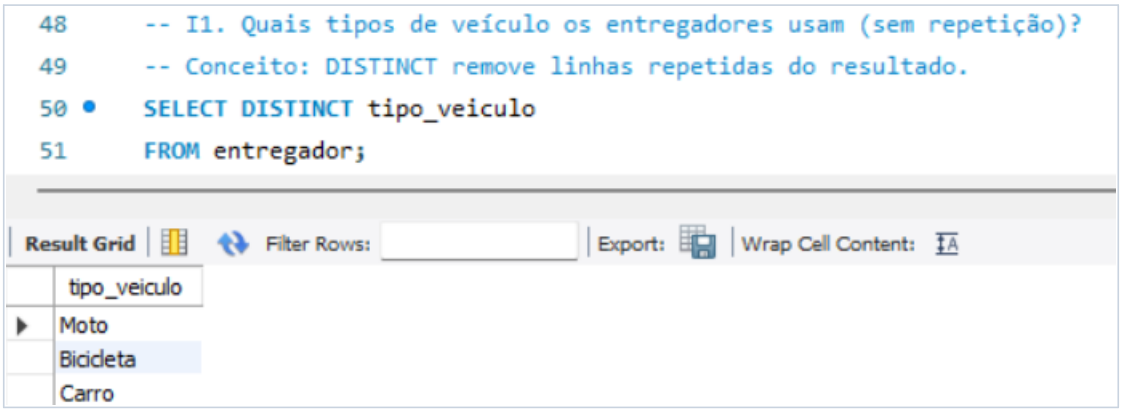


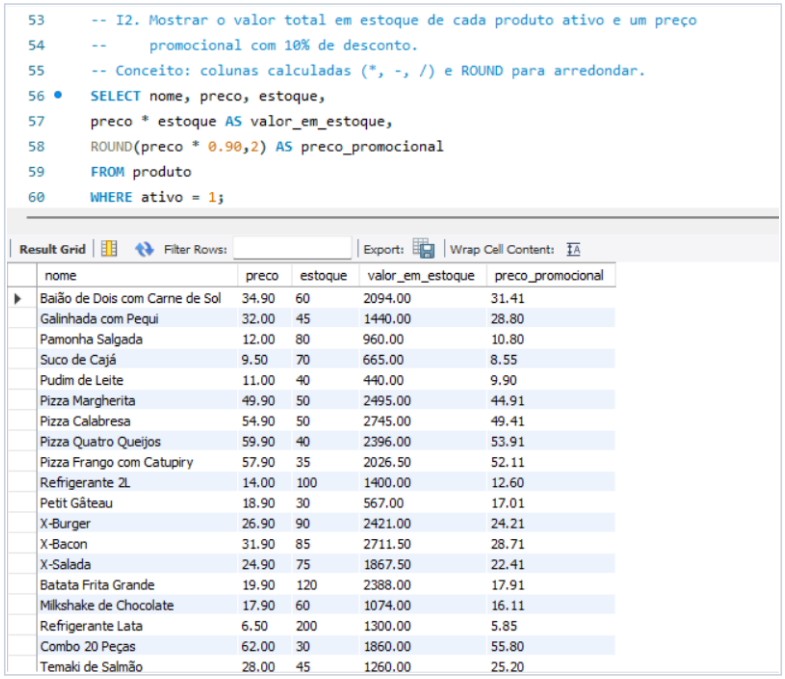
Figura 11 – Resultado da consulta I1 no MySQL Workbench

I2. Mostrar o valor total em estoque de cada produto ativo e um preço promocional com 10% de desconto.

Conceito: colunas calculadas (*, -, /) e ROUND para arredondar.

```
SELECT nome, preco, estoque,
preco * estoque AS valor_em_estoque,
ROUND(preco * 0.90,2) AS preco_promocional
FROM produto
WHERE ativo = 1;
```

Resultado: São listados os 23 produtos ativos (a Feijoada, inativa, fica de fora), com o valor em estoque (preço × estoque) e o preço promocional com 10% de desconto.



nome	preco	estoque	valor_em_estoque	preco_promocional
Baião de Dois com Carne de Sol	34.90	60	2094.00	31.41
Galinhada com Pequi	32.00	45	1440.00	28.80
Pamonha Salgada	12.00	80	960.00	10.80
Suco de Cajá	9.50	70	665.00	8.55
Pudim de Leite	11.00	40	440.00	9.90
Pizza Margherita	49.90	50	2495.00	44.91
Pizza Calabresa	54.90	50	2745.00	49.41
Pizza Quatro Queijos	59.90	40	2396.00	53.91
Pizza Frango com Catupiry	57.90	35	2026.50	52.11
Refrigerante 2L	14.00	100	1400.00	12.60
Petit Gâteau	18.90	30	567.00	17.01
X-Burger	26.90	90	2421.00	24.21
X-Bacon	31.90	85	2711.50	28.71
X-Salada	24.90	75	1867.50	22.41
Batata Frita Grande	19.90	120	2388.00	17.91
Milkshake de Chocolate	17.90	60	1074.00	16.11
Refrigerante Lata	6.50	200	1300.00	5.85
Combo 20 Peças	62.00	30	1860.00	55.80
Temaki de Salmão	28.00	45	1260.00	25.20

Figura 12 – Resultado da consulta I2 no MySQL Workbench

I3. Produtos ativos caros (>= 50) OU com estoque baixo (< 30), exceto bebidas.

Conceito: WHERE com operadores de comparação (>=, <, =) e lógicos (AND, OR, NOT). Os parênteses definem a ordem de avaliação.

```
SELECT nome, preco, estoque, id_categoria
FROM produto
WHERE ativo = 1
AND (preco >= 50 OR estoque < 30)
AND NOT (id_categoria = 6);
```

Resultado: Cinco produtos atendem à regra: as pizzas Calabresa, Quatro Queijos e Frango com Catupiry, o Combo 20 Peças e o Mochi Sorvete (estoque baixo). Nenhuma bebida aparece.

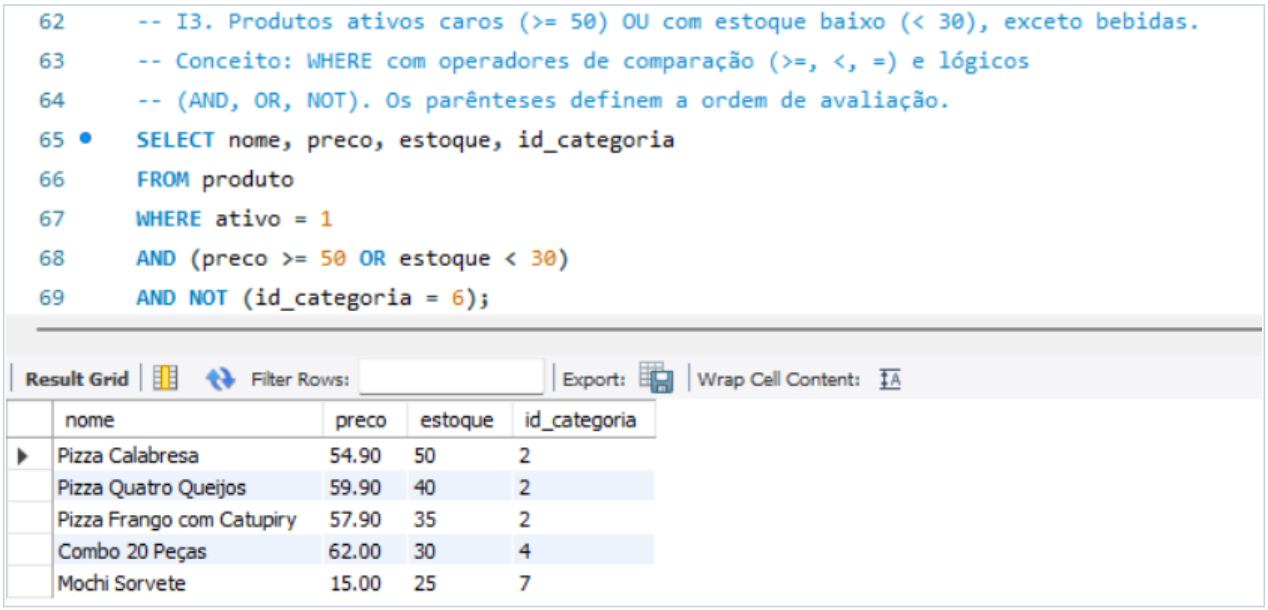


Figura 13 – Resultado da consulta I3 no MySQL Workbench

I4. Buscar produtos de pizza ou lanches "X-".

Conceito: LIKE com curinga % (qualquer sequência de caracteres).

```
SELECT nome, preco
FROM produto
WHERE nome LIKE '%Pizza%'
   OR nome LIKE 'X-%';
```

Resultado: Sete produtos: as quatro pizzas e os três lanches que começam com "X-".

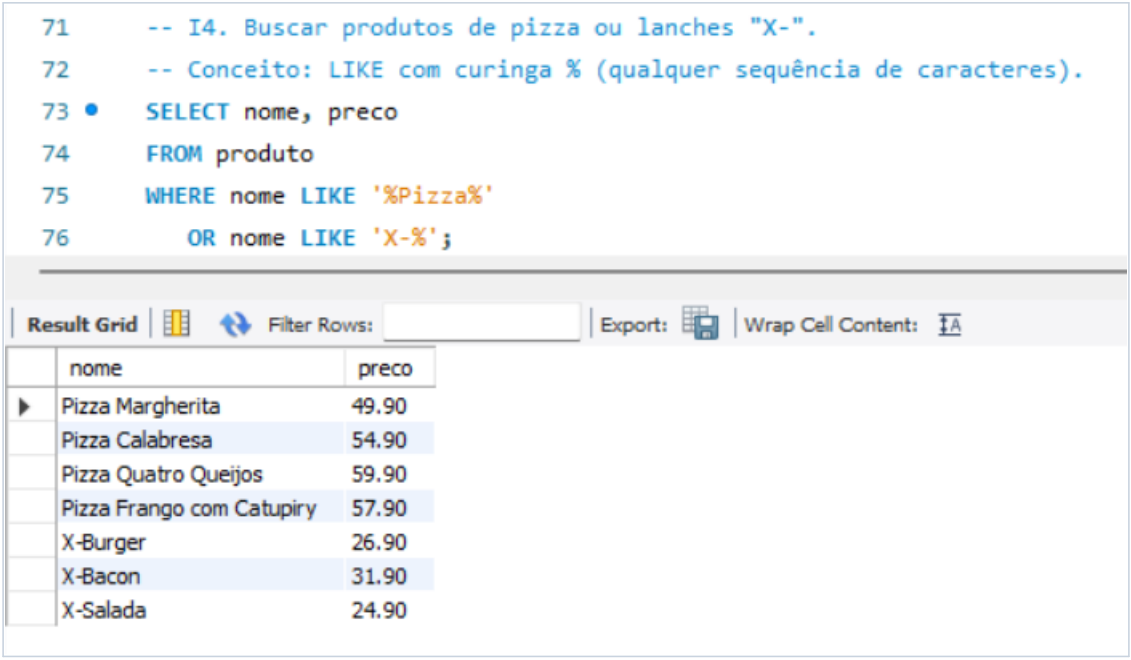


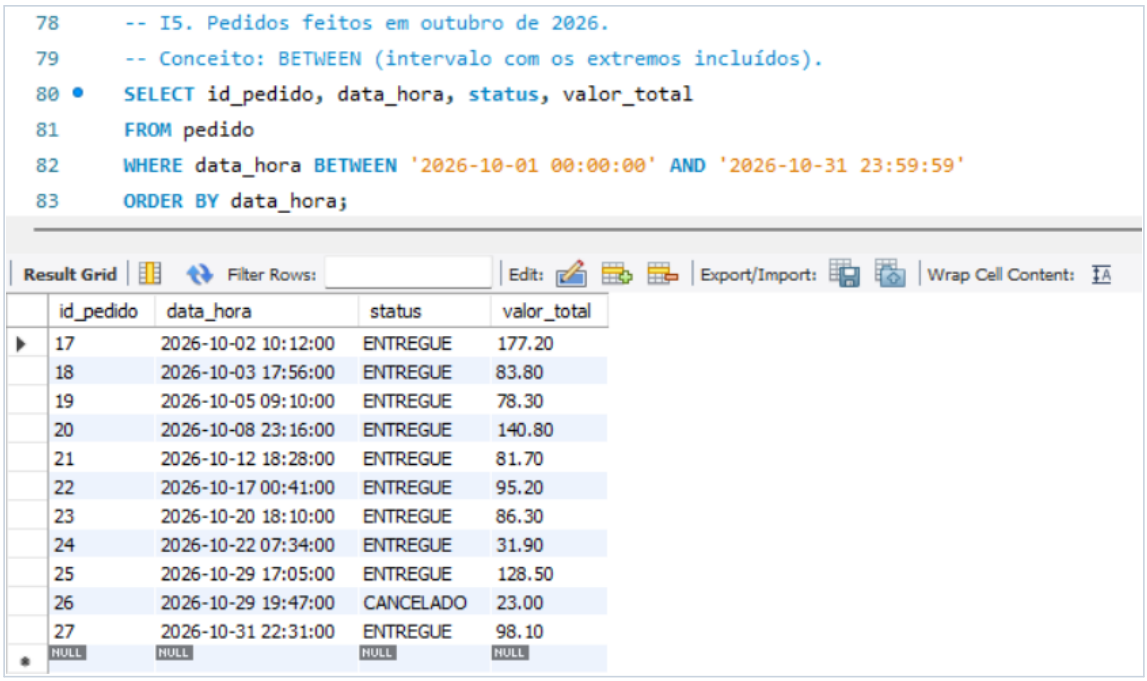
Figura 14 – Resultado da consulta I4 no MySQL Workbench

I5. Pedidos feitos em outubro de 2026.

Conceito: BETWEEN (intervalo com os extremos incluídos).

```
SELECT id_pedido, data_hora, status, valor_total
FROM pedido
WHERE data_hora BETWEEN '2026-10-01 00:00:00' AND '2026-10-31 23:59:59'
ORDER BY data_hora;
```

Resultado: Onze pedidos feitos em outubro de 2026, em ordem de data.



```
78  -- I5. Pedidos feitos em outubro de 2026.
79  -- Conceito: BETWEEN (intervalo com os extremos incluídos).
80  SELECT id_pedido, data_hora, status, valor_total
81  FROM pedido
82  WHERE data_hora BETWEEN '2026-10-01 00:00:00' AND '2026-10-31 23:59:59'
83  ORDER BY data_hora;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_pedido	data_hora	status	valor_total
17	2026-10-02 10:12:00	ENTREGUE	177.20
18	2026-10-03 17:56:00	ENTREGUE	83.80
19	2026-10-05 09:10:00	ENTREGUE	78.30
20	2026-10-08 23:16:00	ENTREGUE	140.80
21	2026-10-12 18:28:00	ENTREGUE	81.70
22	2026-10-17 00:41:00	ENTREGUE	95.20
23	2026-10-20 18:10:00	ENTREGUE	86.30
24	2026-10-22 07:34:00	ENTREGUE	31.90
25	2026-10-29 17:05:00	ENTREGUE	128.50
26	2026-10-29 19:47:00	CANCELADO	23.00
27	2026-10-31 22:31:00	ENTREGUE	98.10
NULL	NULL	NULL	NULL

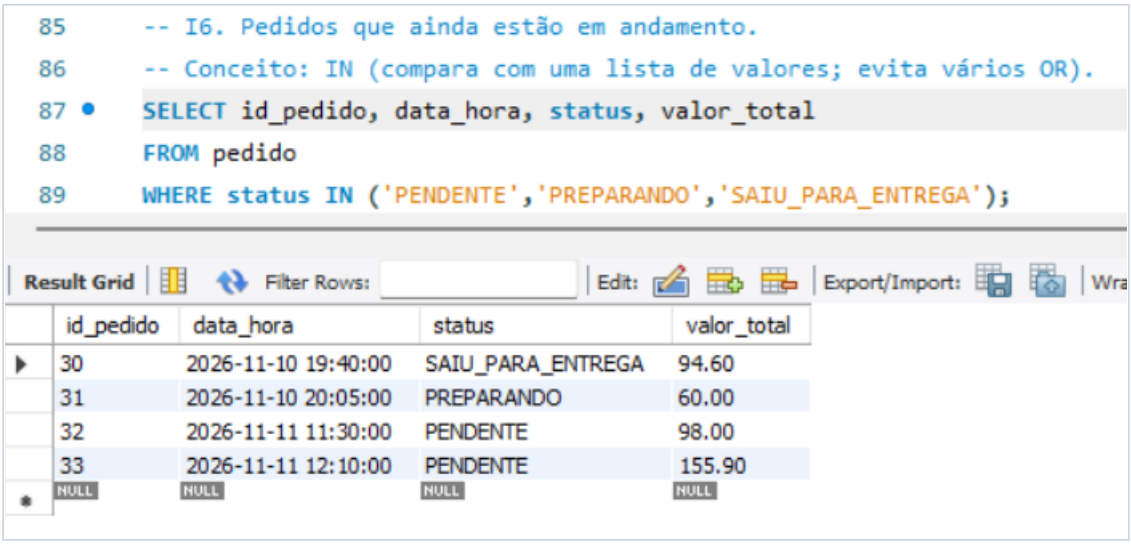
Figura 15 – Resultado da consulta I5 no MySQL Workbench

I6. Pedidos que ainda estão em andamento.

Conceito: IN (compara com uma lista de valores; evita vários OR).

```
SELECT id_pedido, data_hora, status, valor_total
FROM pedido
WHERE status IN ('PENDENTE','PREPARANDO','SAIU_PARA_ENTREGA');
```

Resultado: Quatro pedidos em andamento (ids 30 a 33): um saiu para entrega, um está sendo preparado e dois estão pendentes.



```
85  -- I6. Pedidos que ainda estão em andamento.
86  -- Conceito: IN (compara com uma lista de valores; evita vários OR).
87  SELECT id_pedido, data_hora, status, valor_total
88  FROM pedido
89  WHERE status IN ('PENDENTE','PREPARANDO','SAIU_PARA_ENTREGA');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wra

id_pedido	data_hora	status	valor_total
30	2026-11-10 19:40:00	SAIU_PARA_ENTREGA	94.60
31	2026-11-10 20:05:00	PREPARANDO	60.00
32	2026-11-11 11:30:00	PENDENTE	98.00
33	2026-11-11 12:10:00	PENDENTE	155.90
NULL	NULL	NULL	NULL

Figura 16 – Resultado da consulta I6 no MySQL Workbench

I7. Resumo financeiro dos pedidos entregues: quantidade, faturamento, ticket médio, maior e menor pedido.

Conceito: funções de agregação COUNT, SUM, AVG, MAX e MIN.

```
SELECT COUNT(*) AS qtd_pedidos,
SUM(valor_total) AS faturamento,
ROUND(AVG(valor_total),2) AS ticket_medio,
MAX(valor_total) AS maior_pedido,
MIN(valor_total) AS menor_pedido
FROM pedido
WHERE status = 'ENTREGUE';
```

Resultado: Dos pedidos entregues: 26 pedidos, faturamento de R$ 3.185,50, ticket médio de R$ 122,52, maior pedido de R$ 351,90 e menor de R$ 17,00.

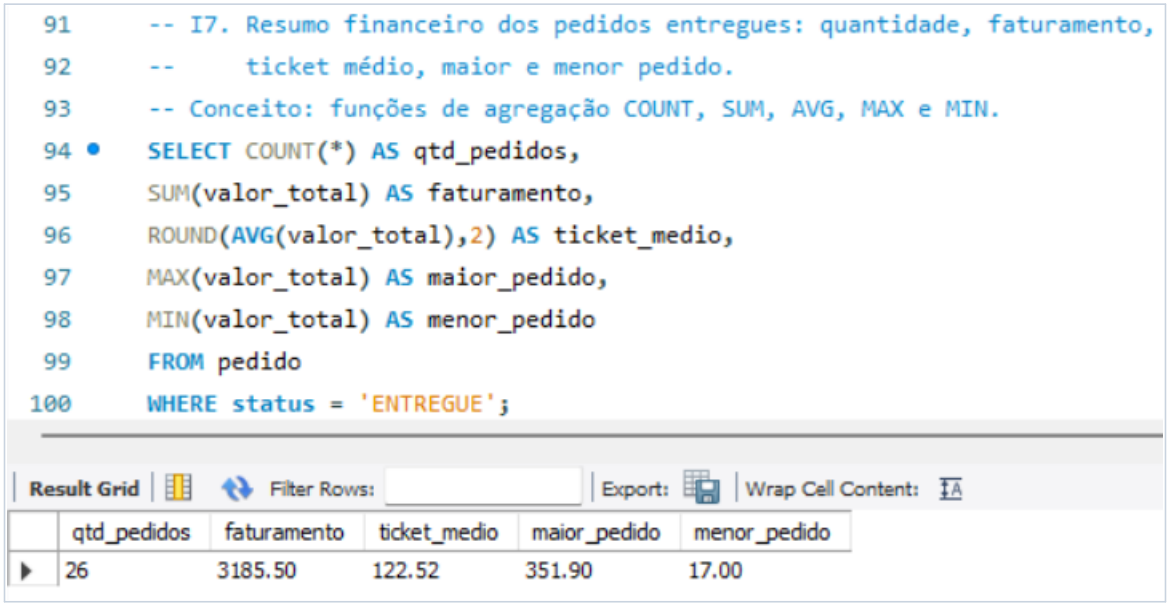


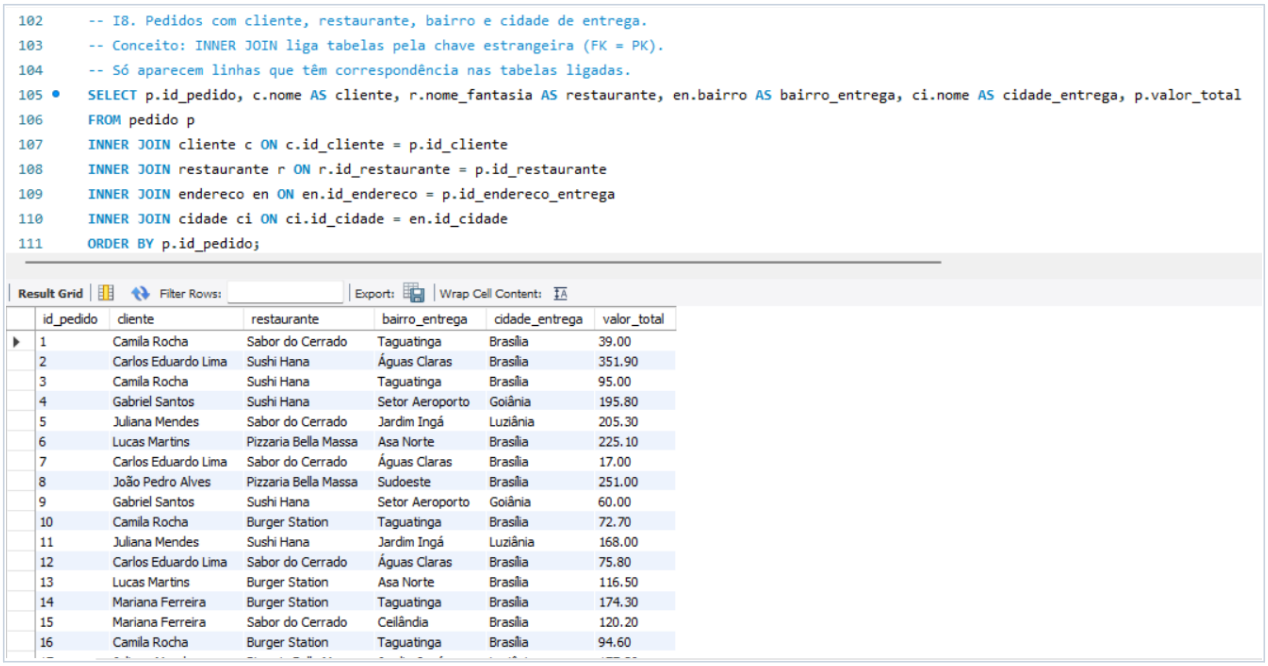
Figura 17 – Resultado da consulta I7 no MySQL Workbench

I8. Pedidos com cliente, restaurante, bairro e cidade de entrega.

Conceito: INNER JOIN liga tabelas pela chave estrangeira (FK = PK). Só aparecem linhas que têm correspondência nas tabelas ligadas.

```
SELECT p.id_pedido, c.nome AS cliente, r.nome_fantasia AS restaurante, en.bairro AS bairro_entrega,
    ci.nome AS cidade_entrega, p.valor_total
FROM pedido p
INNER JOIN cliente c ON c.id_cliente = p.id_cliente
INNER JOIN restaurante r ON r.id_restaurante = p.id_restaurante
INNER JOIN endereco en ON en.id_endereco = p.id_endereco_entrega
INNER JOIN cidade ci ON ci.id_cidade = en.id_cidade
ORDER BY p.id_pedido;
```

Resultado: Os 33 pedidos com o nome do cliente, o restaurante e o bairro e a cidade de entrega, obtidos pela ligação de quatro tabelas.



```
102     -- I8. Pedidos com cliente, restaurante, bairro e cidade de entrega.
103     -- Conceito: INNER JOIN liga tabelas pela chave estrangeira (FK = PK).
104     -- Só aparecem linhas que têm correspondência nas tabelas ligadas.
105 •   SELECT p.id_pedido, c.nome AS cliente, r.nome_fantasia AS restaurante, en.bairro AS bairro_entrega, ci.nome AS cidade_entrega, p.valor_total
106     FROM pedido p
107     INNER JOIN cliente c ON c.id_cliente = p.id_cliente
108     INNER JOIN restaurante r ON r.id_restaurante = p.id_restaurante
109     INNER JOIN endereco en ON en.id_endereco = p.id_endereco_entrega
110     INNER JOIN cidade ci ON ci.id_cidade = en.id_cidade
111     ORDER BY p.id_pedido;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	cliente	restaurante	bairro_entrega	cidade_entrega	valor_total
1	Camila Rocha	Sabor do Cerrado	Taguatinga	Brasília	39.00
2	Carlos Eduardo Lima	Sushi Hana	Águas Claras	Brasília	351.90
3	Camila Rocha	Sushi Hana	Taguatinga	Brasília	95.00
4	Gabriel Santos	Sushi Hana	Setor Aeroporto	Goiânia	195.80
5	Juliana Mendes	Sabor do Cerrado	Jardim Ingá	Luziânia	205.30
6	Lucas Martins	Pizzaria Bella Massa	Asa Norte	Brasília	225.10
7	Carlos Eduardo Lima	Sabor do Cerrado	Águas Claras	Brasília	17.00
8	João Pedro Alves	Pizzaria Bella Massa	Sudoeste	Brasília	251.00
9	Gabriel Santos	Sushi Hana	Setor Aeroporto	Goiânia	60.00
10	Camila Rocha	Burger Station	Taguatinga	Brasília	72.70
11	Juliana Mendes	Sushi Hana	Jardim Ingá	Luziânia	168.00
12	Carlos Eduardo Lima	Sabor do Cerrado	Águas Claras	Brasília	75.80
13	Lucas Martins	Burger Station	Asa Norte	Brasília	116.50
14	Mariana Ferreira	Burger Station	Taguatinga	Brasília	174.30
15	Mariana Ferreira	Sabor do Cerrado	Ceilândia	Brasília	120.20
16	Camila Rocha	Burger Station	Taguatinga	Brasília	94.60

Figura 18 – Resultado da consulta I8 no MySQL Workbench

I9. Os 5 produtos mais vendidos (em unidades), desconsiderando cancelados.

Conceito: GROUP BY agrupa linhas; SUM soma por grupo; ORDER BY ... DESC ordena do maior para o menor; LIMIT corta o resultado.

```sql
SELECT pr.nome AS produto,
SUM(i.quantidade) AS total_vendido
FROM item_pedido i
INNER JOIN produto pr ON pr.id_produto = i.id_produto
INNER JOIN pedido p ON p.id_pedido = i.id_pedido
WHERE p.status <> 'CANCELADO'
GROUP BY pr.id_produto, pr.nome
ORDER BY total_vendido DESC, produto
LIMIT 5;
```

Resultado: Os cinco produtos mais vendidos: X-Salada (11 unidades), Batata Frita Grande e X-Burger (10 cada), Pizza Calabresa (9) e Chá Gelado (8). Em empate, vale a ordem alfabética.

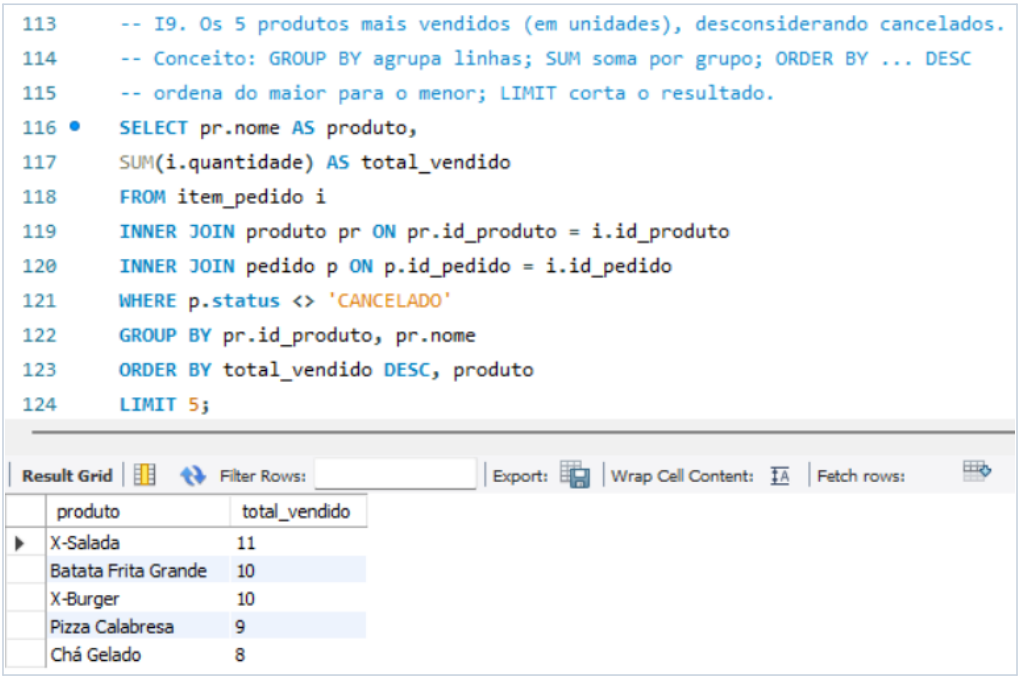


Figura 19 – Resultado da consulta I9 no MySQL Workbench

I10. Restaurantes com faturamento (pedidos entregues) acima de R$ 500.

Conceito: HAVING filtra DEPOIS do agrupamento (WHERE filtra antes e não pode usar SUM/COUNT/AVG).

```
SELECT r.nome_fantasia AS restaurante,
COUNT(p.id_pedido) AS qtd_pedidos,
SUM(p.valor_total) AS faturamento
FROM restaurante r
INNER JOIN pedido p ON p.id_restaurante = r.id_restaurante
WHERE p.status = 'ENTREGUE'
GROUP BY r.id_restaurante, r.nome_fantasia
HAVING SUM(p.valor_total) > 500
ORDER BY faturamento DESC;
```

Resultado: Três restaurantes passam de R$ 500: Pizzaria Bella Massa (R$ 952,10), Sushi Hana (R$ 899,50) e Burger Station (R$ 876,60). O Sabor do Cerrado (R$ 457,30) é filtrado pelo HAVING.

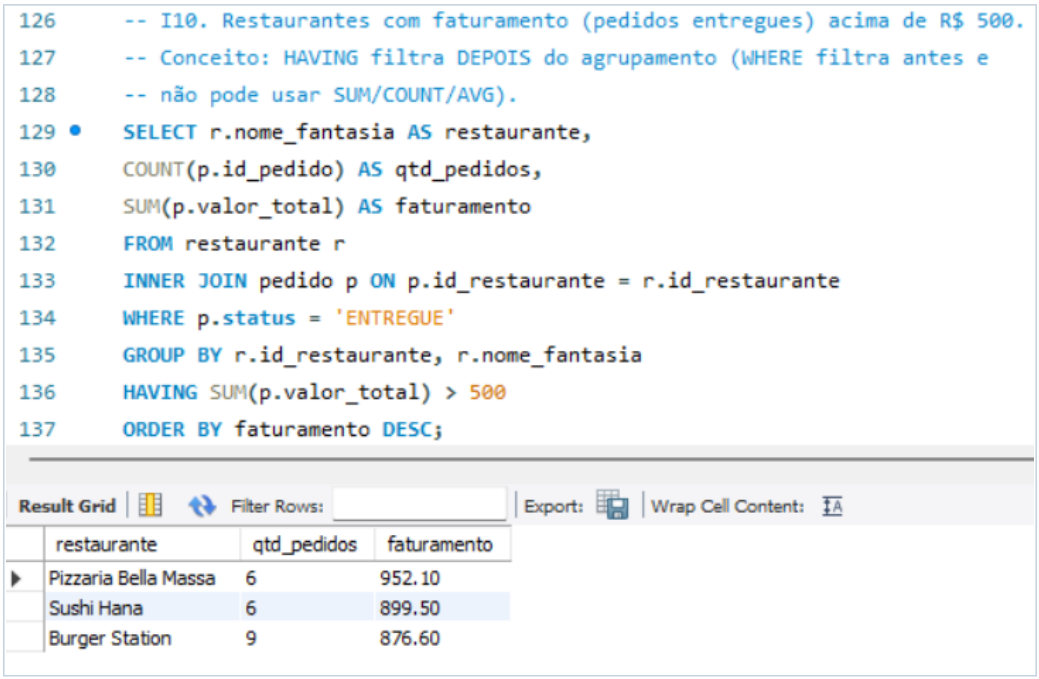


Figura 20 – Resultado da consulta I10 no MySQL Workbench

I11. Nota média e quantidade de avaliações de cada restaurante.

Conceito: AVG com JOIN de 3 tabelas (avaliacao -> pedido -> restaurante).

```
SELECT r.nome_fantasia AS restaurante,
COUNT(a.id_avaliacao) AS qtd_avaliacoes,
ROUND(AVG(a.nota),2) AS nota_media
FROM avaliacao a
INNER JOIN pedido p ON p.id_pedido = a.id_pedido
INNER JOIN restaurante r ON r.id_restaurante = p.id_restaurante
GROUP BY r.id_restaurante, r.nome_fantasia
ORDER BY nota_media DESC;
```

Resultado: Notas médias: Sabor do Cerrado 4,00 (3 avaliações), Burger Station 4,00 (7), Pizzaria Bella Massa 3,50 (4) e Sushi Hana 3,40 (5).

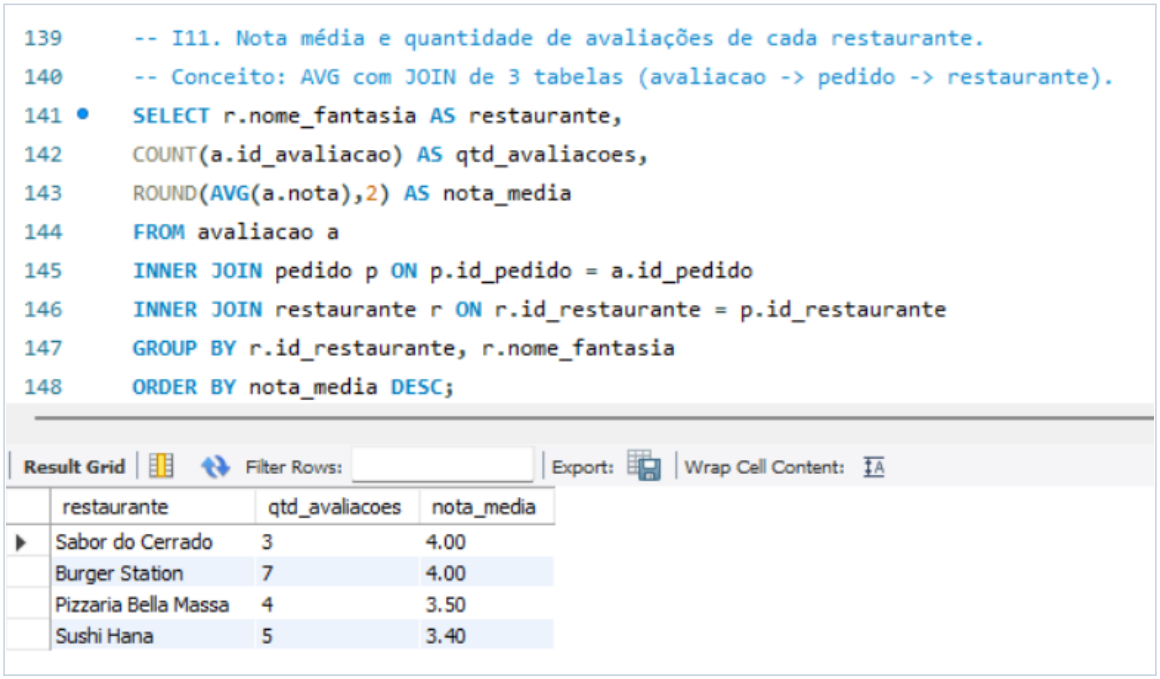


Figura 21 – Resultado da consulta I11 no MySQL Workbench

I12. Produtos ativos com preço acima da média de preços.

Conceito: SUBCONSULTA escalar (retorna um único valor) dentro do WHERE.

```
SELECT nome, preco
FROM produto
WHERE ativo = 1 AND preco > (SELECT AVG(preco) FROM produto WHERE ativo = 1)
ORDER BY preco DESC;
```

Resultado: A média de preços dos produtos ativos é R$ 28,77; dez produtos custam mais que isso, do mais caro (Combo 20 Peças, R$ 62,00) ao mais barato (Hot Roll, R$ 29,90).

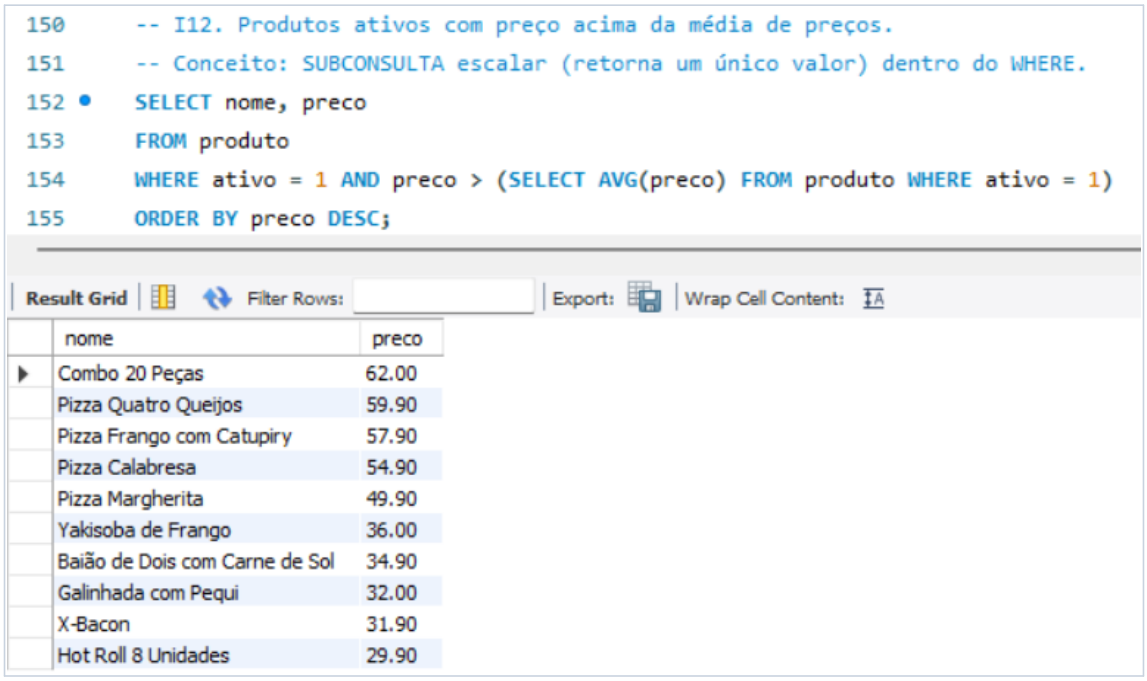


Figura 22 – Resultado da consulta I12 no MySQL Workbench

I13. Clientes que nunca fizeram um pedido.

Conceito: SUBCONSULTA com NOT IN (a lista vem de outra consulta).

```
SELECT id_cliente, nome, data_cadastro
FROM cliente
WHERE id_cliente NOT IN (SELECT id_cliente FROM pedido);
```

Resultado: Patrícia Nogueira e Bruno Carvalho, cadastrados recentemente, ainda não fizeram nenhum pedido.

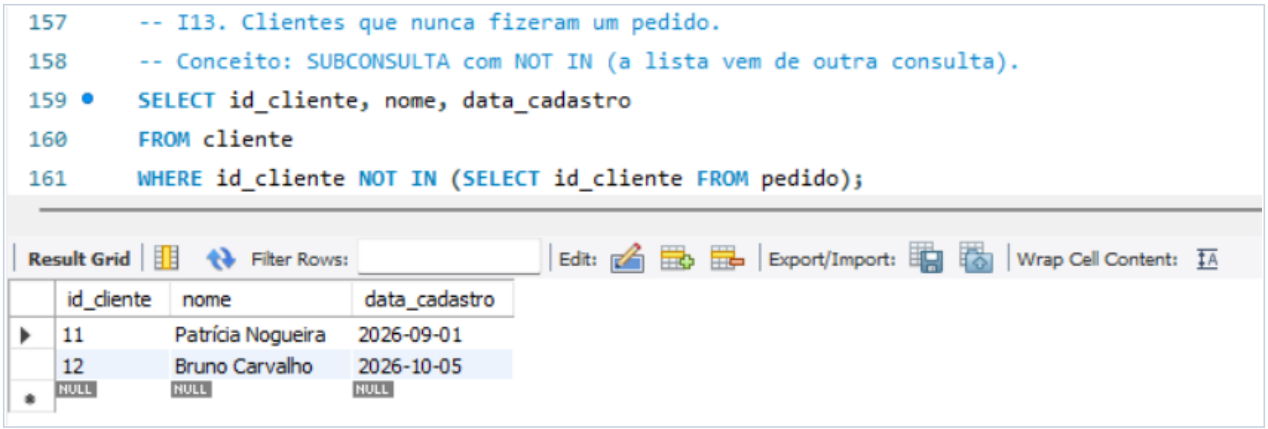


Figura 23 – Resultado da consulta I13 no MySQL Workbench

I14. Quantidade de entregas concluídas por entregador, incluindo quem nunca entregou.

Conceito: LEFT JOIN mantém TODAS as linhas da tabela da esquerda (entregador), mesmo sem correspondência. COUNT(p.id_pedido) ignora NULL, então quem nunca entregou aparece com 0.

```
SELECT e.nome AS entregador, e.tipo_veiculo, e.ativo,
COUNT(p.id_pedido) AS entregas
FROM entregador e
LEFT JOIN pedido p
ON p.id_entregador = e.id_entregador
AND p.status = 'ENTREGUE'
GROUP BY e.id_entregador, e.nome, e.tipo_veiculo, e.ativo
ORDER BY entregas DESC;
```

Resultado: Entregas concluídas por entregador. Felipe Araújo aparece com 0 por causa do LEFT JOIN; Leandro Cunha está inativo, mas tem entregas antigas.

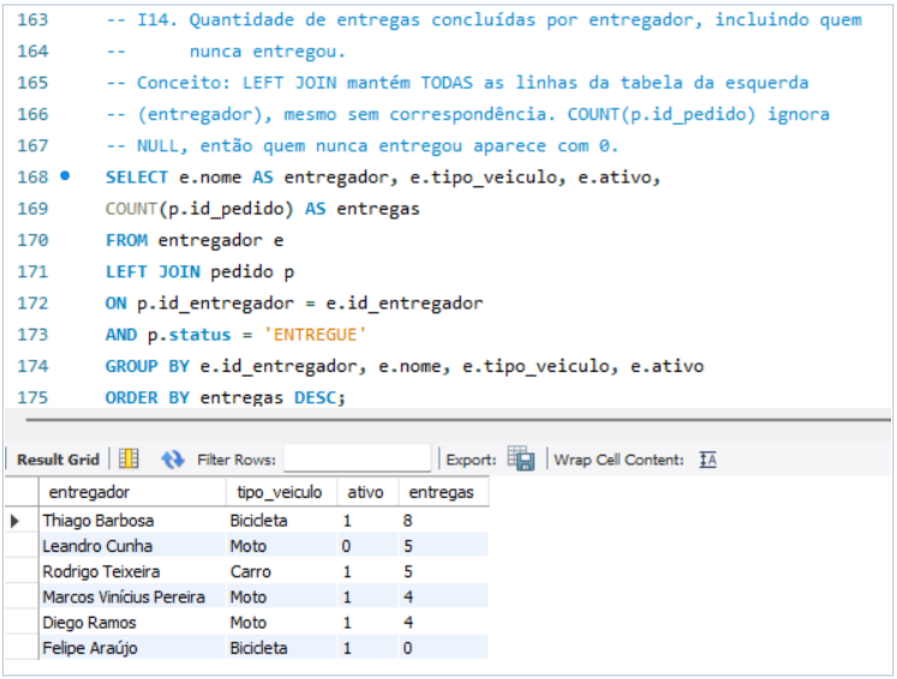


Figura 24 – Resultado da consulta I14 no MySQL Workbench

I15. Agenda de contatos: nome e telefone de clientes e entregadores.

Conceito: UNION junta os resultados de duas consultas (mesmo número de colunas e tipos compatíveis) e remove duplicados.

```
SELECT nome, telefone, 'Cliente' AS tipo FROM cliente
UNION
SELECT nome, telefone, 'Entregador' AS tipo FROM entregador
ORDER BY nome;
```

Resultado: Agenda unificada com os 12 clientes e os 6 entregadores (18 contatos), ordenada por nome.

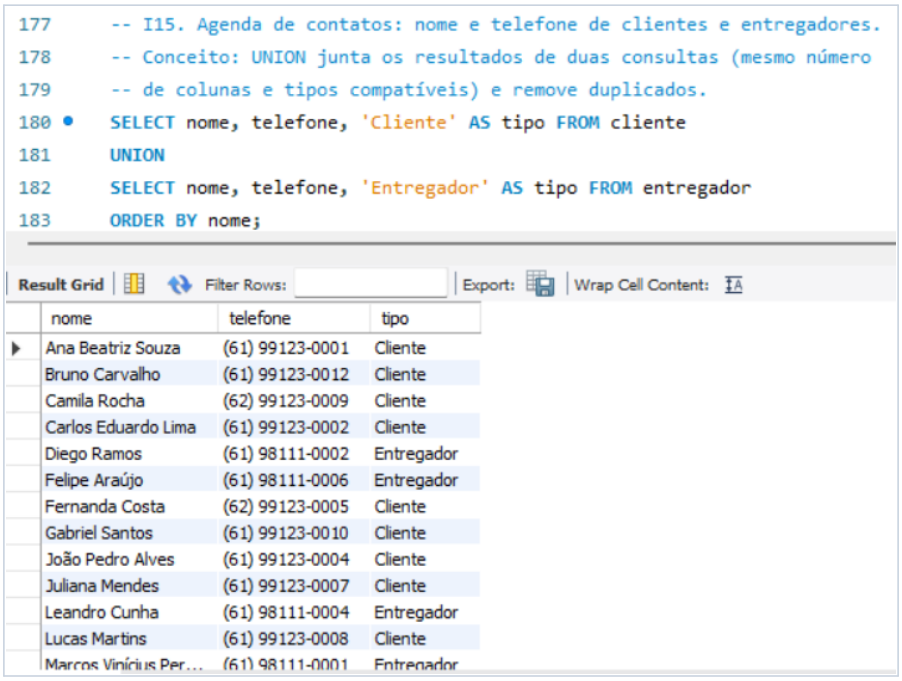


Figura 25 – Resultado da consulta I15 no MySQL Workbench

4.3 Consultas avançadas: Triggers e Stored Procedures

Trigger (gatilho) é um código que o MySQL executa automaticamente quando ocorre um INSERT, UPDATE ou DELETE em uma tabela, antes (*BEFORE*) ou depois (*AFTER*) da operação. Dentro dele, *NEW.coluna* é o valor novo e *OLD.coluna* é o valor antigo. **Stored Procedure** é um bloco de código salvo no banco, executado quando chamado com *CALL*, e que pode receber parâmetros.

Como esses objetos têm vários comandos terminados em ponto e vírgula, usamos **DELIMITER $$** durante a criação para que o MySQL não interprete o primeiro ";" como o fim do objeto; ao final voltamos com **DELIMITER ;**. O comando **SIGNAL SQLSTATE '45000'** interrompe a operação e devolve uma mensagem de erro personalizada.

```
DELIMITER $$
-- ... criação de triggers e procedures (A1 a A5) ...
DELIMITER ;
```

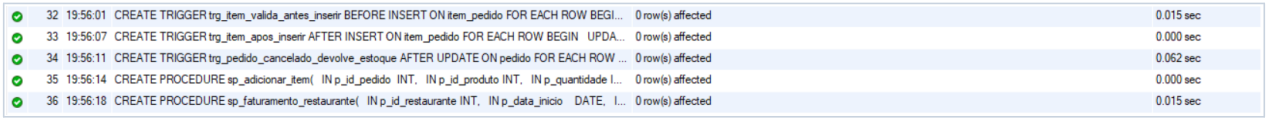


Figura 26 – Criação dos 3 triggers e das 2 procedures sem erros

A1. Trigger *trg_item_valida_antes_inserir* (BEFORE INSERT em item_pedido)

Valida o item **antes** de ele ser gravado. Três regras: o produto precisa estar ativo, pertencer ao restaurante do pedido e ter estoque suficiente. Se alguma falhar, o SIGNAL cancela o INSERT e mostra a mensagem de erro.

```
CREATE TRIGGER trg_item_valida_antes_inserir
BEFORE INSERT ON item_pedido
FOR EACH ROW
BEGIN
  DECLARE v_estoque INT;
  DECLARE v_ativo TINYINT;
  DECLARE v_rest_produto INT;
  DECLARE v_rest_pedido INT;

  SELECT estoque, ativo, id_restaurante
    INTO v_estoque, v_ativo, v_rest_produto
  FROM produto
  WHERE id_produto = NEW.id_produto;

  SELECT id_restaurante INTO v_rest_pedido
  FROM pedido
  WHERE id_pedido = NEW.id_pedido;

  IF v_ativo = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Produto inativo: nao pode ser vendido.';
  END IF;

  IF v_rest_produto <> v_rest_pedido THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'O produto nao pertence ao restaurante do pedido.';
  END IF;

  IF v_estoque < NEW.quantidade THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Estoque insuficiente para este produto.';
  END IF;
END$$
```

A2. Trigger *trg_item_apos_inserir* (AFTER INSERT em item_pedido)

Depois de gravar o item, **dá baixa no estoque** do produto e **recalcula o valor_total** do pedido (soma dos itens + taxa de entrega).

```
CREATE TRIGGER trg_item_apos_inserir
AFTER INSERT ON item_pedido
FOR EACH ROW
BEGIN
  UPDATE produto
     SET estoque = estoque - NEW.quantidade
   WHERE id_produto = NEW.id_produto;

  UPDATE pedido
     SET valor_total = taxa_entrega +
         (SELECT SUM(quantidade * preco_unitario)
            FROM item_pedido
           WHERE id_pedido = NEW.id_pedido)
   WHERE id_pedido = NEW.id_pedido;
END$$
```

A3. Trigger *trg_pedido_cancelado_devolve_estoque* (AFTER UPDATE em pedido)

Quando o status muda para CANCELADO, **devolve ao estoque** as quantidades dos itens do pedido. A condição *OLD.status <> NEW.status* evita devolver duas vezes se o pedido já estava cancelado.

```
CREATE TRIGGER trg_pedido_cancelado_devolve_estoque
AFTER UPDATE ON pedido
FOR EACH ROW
BEGIN
  IF NEW.status = 'CANCELADO' AND OLD.status <> 'CANCELADO' THEN
    UPDATE produto pr
    INNER JOIN item_pedido i ON i.id_produto = pr.id_produto
       SET pr.estoque = pr.estoque + i.quantidade
     WHERE i.id_pedido = NEW.id_pedido;
  END IF;
END$$
```

A4. Procedure *sp_adicionar_item*

Adiciona um produto a um pedido. Busca o preço atual do produto e insere o item com esse preço; os triggers A1 e A2 validam o item e atualizam estoque e total automaticamente. Parâmetros IN: pedido, produto e quantidade.

```
CREATE PROCEDURE sp_adicionar_item(
  IN p_id_pedido  INT,
  IN p_id_produto INT,
  IN p_quantidade INT
)
BEGIN
  DECLARE v_preco DECIMAL(10,2);

  SELECT preco INTO v_preco
  FROM produto
  WHERE id_produto = p_id_produto;

  IF v_preco IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Produto nao encontrado.';
  END IF;

  INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario)
  VALUES (p_id_pedido, p_id_produto, p_quantidade, v_preco);
END$$
```

A5. Procedure *sp_faturamento_restaurante*

Relatório de um restaurante em um período (só pedidos ENTREGUES): quantidade de pedidos, faturamento e ticket médio. O COALESCE troca NULL por 0 quando não há pedidos no período.

```
CREATE PROCEDURE sp_faturamento_restaurante(
  IN p_id_restaurante INT,
  IN p_data_inicio    DATE,
  IN p_data_fim       DATE
)
BEGIN
  SELECT r.nome_fantasia AS restaurante,
         COUNT(p.id_pedido)                          AS qtd_pedidos,
         COALESCE(SUM(p.valor_total),0)              AS faturamento,
         COALESCE(ROUND(AVG(p.valor_total),2),0)     AS ticket_medio
  FROM restaurante r
  LEFT JOIN pedido p
         ON p.id_restaurante = r.id_restaurante
        AND p.status = 'ENTREGUE'
        AND DATE(p.data_hora) BETWEEN p_data_inicio AND p_data_fim
  WHERE r.id_restaurante = p_id_restaurante
  GROUP BY r.id_restaurante, r.nome_fantasia;
END$$
```

Testes das consultas avançadas

Os testes foram feitos dentro de uma transação terminada em **ROLLBACK**: os comandos rodam e mostram o efeito, mas os dados voltam ao estado original. Os resultados mostram os triggers e as procedures funcionando.

Teste A5 – relatório de faturamento

```
CALL sp_faturamento_restaurante(4, '2026-10-01', '2026-10-31');
```

O Sushi Hana (id 4) teve 2 pedidos entregues em outubro, somando R$ 224,60 (ticket médio de R$ 112,30).

```
331 ●  -- ======================================================================
332    -- TESTES DAS CONSULTAS AVANÇADAS
333    -- Cada teste fica dentro de uma TRANSAÇÃO que termina com ROLLBACK:
334    -- os comandos rodam e mostram o resultado, mas os dados voltam ao que
335    -- eram antes (nada é gravado de verdade).
336    -- ----------------------------------------------------------------------
337
338    -- Teste A5 (procedure de relatório): faturamento do Sushi Hana (id 4) em outubro.
339    CALL sp_faturamento_restaurante(4, '2026-10-01', '2026-10-31');
340
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurante	qtd_pedidos	faturamento	ticket_medio
Sushi Hana	2	224.60	112.30

Figura 27 – Teste da procedure sp_faturamento_restaurante

Teste A4 + A2 – procedure de item e trigger de estoque e total

```
START TRANSACTION;
  SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;           -- antes
  SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- antes
  CALL sp_adicionar_item(33, 22, 2);
  SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;           -- depois
  SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- depois
ROLLBACK;
```

Foram adicionadas 2 unidades do Yakisoba de Frango (produto 22) ao pedido 33. O trigger baixou o estoque de **35 para 33** e recalculou o valor_total do pedido de **R$ 155,90 para R$ 227,90** (+ 2 × R$ 36,00 = R$ 72,00).

```
341    -- Teste A4 + A2 (procedure de item + trigger de estoque e total):
342    -- adiciona 2 unidades do Yakisoba (produto 22) ao pedido 33 (Sushi Hana).
343 ●  START TRANSACTION;
344 ●    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- antes
345 ●    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- antes
346 ●    CALL sp_adicionar_item(33, 22, 2);
347 ●    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- depois: estoque -2
348 ●    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- depois: total + 72,00
349 ●  ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_produto	nome	estoque
22	Yakisoba de Frango	35
NULL	NULL	NULL

Figura 28 – Antes: estoque do produto 22 = 35

```
341     -- Teste A4 + A2 (procedure de item + trigger de estoque e total):
342     -- adiciona 2 unidades do Yakisoba (produto 22) ao pedido 33 (Sushi Hana).
343 •  START TRANSACTION;
344 •    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- antes
345 •    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- antes
346 •    CALL sp_adicionar_item(33, 22, 2);
347 •    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- depois: estoque -2
348 •    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- depois: total + 72,00
349 •  ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_pedido	taxa_entrega	valor_total
33	8.00	155.90
NULL	NULL	NULL

Figura 29 – Antes: valor_total do pedido 33 = 155,90

```
341     -- Teste A4 + A2 (procedure de item + trigger de estoque e total):
342     -- adiciona 2 unidades do Yakisoba (produto 22) ao pedido 33 (Sushi Hana).
343 •  START TRANSACTION;
344 •    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- antes
345 •    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- antes
346 •    CALL sp_adicionar_item(33, 22, 2);
347 •    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- depois: estoque -2
348 •    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- depois: total + 72,00
349 •  ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_produto	nome	estoque
22	Yakisoba de Frango	33
NULL	NULL	NULL

Figura 30 – Depois: estoque do produto 22 = 33 (baixa de 2 unidades)

```
341     -- Teste A4 + A2 (procedure de item + trigger de estoque e total):
342     -- adiciona 2 unidades do Yakisoba (produto 22) ao pedido 33 (Sushi Hana).
343 •  START TRANSACTION;
344 •    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- antes
345 •    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- antes
346 •    CALL sp_adicionar_item(33, 22, 2);
347 •    SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- depois: estoque -2
348 •    SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- depois: total + 72,00
349 •  ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_pedido	taxa_entrega	valor_total
33	8.00	227.90
NULL	NULL	NULL

Figura 31 – Depois: valor_total do pedido 33 = 227,90

Teste A3 – cancelamento devolve o estoque

```
START TRANSACTION;
  SELECT i.id_produto, pr.nome, pr.estoque AS estoque_antes
  FROM item_pedido i INNER JOIN produto pr ON pr.id_produto = i.id_produto
  WHERE i.id_pedido = 33;
  UPDATE pedido SET status = 'CANCELADO' WHERE id_pedido = 33;
  SELECT i.id_produto, pr.nome, pr.estoque AS estoque_depois, i.quantidade AS devolvido
  FROM item_pedido i INNER JOIN produto pr ON pr.id_produto = i.id_produto
  WHERE i.id_pedido = 33;
ROLLBACK;
```

Ao cancelar o pedido 33, o trigger devolveu ao estoque as quantidades dos três itens: Combo 20 Peças (30 → 31), Temaki de Salmão (45 → 47) e Hot Roll 8 Unidades (40 → 41).

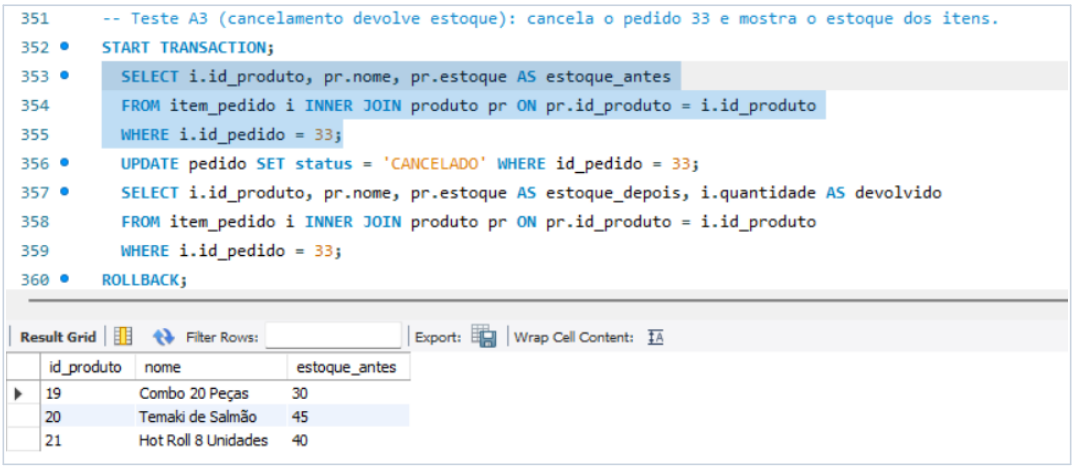


Figura 32 – Antes do cancelamento

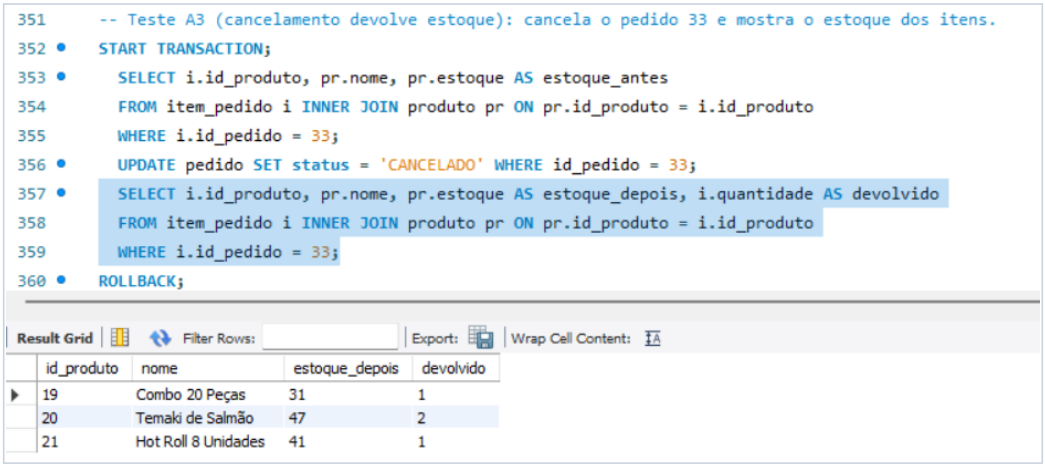


Figura 33 – Depois do cancelamento: estoque devolvido

Teste A1 – validações do trigger

Cada chamada abaixo viola uma regra do trigger A1 (ou da procedure A4) e é bloqueada com o erro 1644 (SQLSTATE 45000) e a mensagem correspondente:

Chamada	Regra violada	Mensagem retornada
`CALL sp_adicionar_item(33, 3, 1);`	Produto inativo (Feijoada Individual)	Produto inativo: nao pode ser vendido.
`CALL sp_adicionar_item(33, 7, 1);`	Produto de outro restaurante (pizza em pedido de sushi)	O produto nao pertence ao restaurante do pedido.
`CALL sp_adicionar_item(33, 19, 999);`	Quantidade maior que o estoque	Estoque insuficiente para este produto.
`CALL sp_adicionar_item(33, 99, 1);`	Produto inexistente	Produto nao encontrado.

5. Considerações Finais

O projeto percorreu todas as etapas do desenvolvimento de um banco de dados relacional: a descrição do cenário e o MER, o DER com a normalização e as regras de integridade, a implementação física em MySQL e a exploração dos dados com 26 consultas. As decisões de modelagem mais importantes foram separar endereço e cidade, tratar o relacionamento entre cliente e endereço como N:N, guardar o preço unitário no item do pedido e usar triggers para manter estoque e valor total coerentes.

Na prática, o grupo aprendeu que o modelo precisa ser conferido em cada etapa (as cardinalidades do MER devem aparecer corretamente no DER), que a ordem de criação e de inserção respeita as chaves estrangeiras e que regras de negócio (como estoque e totais) ficam mais seguras quando o próprio banco as aplica.

Requisitos atendidos

Requisito do projeto	Onde está	Pontos
1.1 Descrição do cenário proposto	Seção 1.1	0,3
1.2 MER no BRModelo	Seção 1.2	
2.1 DER no MySQL Workbench (Data Modeling)	Seção 2.1	0,4
2.2 Explicação detalhada das etapas da iteração	Seção 2.2	
3.1 Script SQL com explicação do código (SQL Development)	Seção 3	0,3
4. Consultas SQL: 4.1 básicas, 4.2 intermediárias, 4.3 avançadas	Seção 4 (26 consultas)	1,0
Total		**2,0**

Arquivos do projeto

Arquivo	Descrição
delivery_restaurante_conceitual.brM3 / .png	MER no BRModelo
delivery_restaurante_logico.mwb / .png	DER no MySQL Workbench
01_schema_delivery.sql	Script DDL (banco e tabelas)
02_dados_delivery.sql	Script DML (dados de exemplo)
03_consultas_delivery.sql	Consultas, triggers e procedures
Projeto_Final_BD_Delivery_Restaurante.pdf	Este documento (os scripts completos estão nos Apêndices A, B e C)

Referências

- Programadores Deprê. *Comandos SQL: DDL, DML, DCL, TCL, DQL e Cláusulas*. https://programadoresdepre.com.br/comandos-sql-ddl-dml-dcl-tcl-dql-e-clausulas/
- W3Schools. *SQL Tutorial*. https://www.w3schools.com/sql/
- Oracle. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/
- Material da disciplina Modelagem de Banco de Dados, Profa. MsC Josyane Lannes Florenzano de Souza (UDF).

Apêndice A – Script 01_schema_delivery.sql (completo)

Script DDL completo: cria o banco *delivery_restaurante* e as 13 tabelas, com as explicações em comentário. As linhas estão numeradas para facilitar a referência.

```
1  -- ========================================================================
2  -- PROJETO FINAL - MODELAGEM DE BANCO DE DADOS
3  -- Tema: Delivery de Restaurante
4  -- Modelo físico (script DDL) - 13 tabelas - MySQL 8
5
6  -- Grupo:
7  -- -> Bernardo Kopp Pinheiro
8  -- -> Gabriel Valois Rodrigues
9  -- -> Pedro Henrique Lima dos Santos Mota
10 -- -> Piêtro Bitencourt Nunes
11 -- -> Zamorano Fragoso de Sousa
12
13 -- LEGENDA RAPIDA (detalhes em cada tabela):
14 --   DDL ............ Data Definition Language: comandos que DEFINEM a
15 --                    estrutura (CREATE, ALTER, DROP).
16 --   PRIMARY KEY .... chave primária: identifica cada linha de forma única.
17 --   FOREIGN KEY .... chave estrangeira: liga uma coluna à PK de outra
18 --                    tabela e impede valores que não existem lá.
19 --   UNIQUE ......... não permite dois valores iguais na coluna.
20 --   NOT NULL ....... a coluna é obrigatória.
21 --   DEFAULT ........ valor usado quando nenhum é informado no INSERT.
22 --   CHECK .......... regra de validação (ex.: preço não pode ser negativo).
23 -- ========================================================================
24
25
26 -- ------------------------------------------------------------------------
27 -- DROP DATABASE IF EXISTS:
28 --   apaga o banco "delivery_restaurante" caso ele já exista. Serve para
29 --   poder rodar o script várias vezes, sempre recomeçando do zero.
30 --   (CUIDADO: apaga todos os dados desse banco.)
31 -- ------------------------------------------------------------------------
32 DROP DATABASE IF EXISTS delivery_restaurante;
33
34 -- ------------------------------------------------------------------------
35 -- CREATE DATABASE:
36 --   cria o banco de dados.
37 --   DEFAULT CHARACTER SET utf8mb4 -> conjunto de caracteres que aceita
38 --       acentos (ç, ã, é...) e até emojis.
39 --   DEFAULT COLLATE utf8mb4_unicode_ci -> regra de comparação e ordenação
40 --       de textos. "ci" = case insensitive (não diferencia maiúscula de
41 --       minúscula: 'ana' = 'ANA').
42 -- ------------------------------------------------------------------------
43 CREATE DATABASE delivery_restaurante
44   DEFAULT CHARACTER SET utf8mb4
45   DEFAULT COLLATE utf8mb4_unicode_ci;
46
47 -- USE: seleciona o banco em que os próximos comandos serão executados.
48 USE delivery_restaurante;
49
50
51 -- ------------------------------------------------------------------------
52 -- CIDADE
53 -- Guarda as cidades separadas do endereço, para que o registro da cidade
54 -- não se perca se um endereço for excluído (decisão tomada na modelagem).
55 -- ------------------------------------------------------------------------
56 CREATE TABLE cidade (
57 id_cidade INT NOT NULL AUTO_INCREMENT,
58 nome VARCHAR(100) NOT NULL,
59 uf CHAR(2) NOT NULL,
60 PRIMARY KEY (id_cidade),
61 UNIQUE KEY uq_cidade_nome_uf (nome, uf)
62 -- UNIQUE KEY composta: o par (nome, uf) não pode se repetir, ou seja,
63 -- não existem duas vezes "Goiânia/GO". Mas "Goiânia/GO" e "Goiânia/XX" seriam aceitas.
64 ) ENGINE=InnoDB;
65
66 -- ENGINE=InnoDB: motor de armazenamento da tabela. O InnoDB é o motor
67 -- padrão do MySQL e é o que SUPORTA chave estrangeira (FOREIGN KEY),
68 -- transações (COMMIT/ROLLBACK) e bloqueio por linha. Motores antigos,
69 -- como o MyISAM, ignoram as chaves estrangeiras. Por isso usamos
70 -- InnoDB em todas as tabelas.
71
72 -- ------------------------------------------------------------------------
73 -- ENDERECO (N:1 com CIDADE)
```

```sql
-- Endereço reutilizável: serve para clientes, restaurantes e entregas.
-- ---------------------------------------------------------------------
CREATE TABLE endereco (
id_endereco INT NOT NULL AUTO_INCREMENT,
logradouro VARCHAR(150) NOT NULL,
numero VARCHAR(10) NOT NULL,    -- VARCHAR porque pode ter letras (ex.: "12A", "S/N")
complemento VARCHAR(60) NULL,   -- NULL = campo opcional
bairro VARCHAR(80) NOT NULL,
cep CHAR(8) NOT NULL,           -- CEP sem hífen: sempre 8 dígitos
id_cidade INT NOT NULL,          -- FK para cidade
PRIMARY KEY (id_endereco),
CONSTRAINT fk_endereco_cidade
FOREIGN KEY (id_cidade) REFERENCES cidade (id_cidade)
) ENGINE=InnoDB;
-- CONSTRAINT nome: dá um nome à regra, facilitando achar o erro depois.
-- FOREIGN KEY (...) REFERENCES tabela (coluna): só aceita um id_cidade
-- que já exista em cidade. Sem ON DELETE, vale o comportamento padrão
-- (RESTRICT): o MySQL NÃO deixa excluir uma cidade que ainda tem endereços.

-- ---------------------------------------------------------------------
-- CLIENTE
-- ---------------------------------------------------------------------
CREATE TABLE cliente (
id_cliente INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
cpf CHAR(11) NOT NULL,   -- CPF sem pontos e traço: 11 dígitos
telefone VARCHAR(15)  NULL,
email VARCHAR(100) NOT NULL,
-- DATE: guarda somente a data (AAAA-MM-DD).
data_cadastro DATE NOT NULL,
PRIMARY KEY (id_cliente),
-- UNIQUE: impede dois clientes com o mesmo CPF ou o mesmo e-mail.
UNIQUE KEY uq_cliente_cpf (cpf),
UNIQUE KEY uq_cliente_email (email)
) ENGINE=InnoDB;


-- ---------------------------------------------------------------------
-- CLIENTE_ENDERECO
-- Tabela ASSOCIATIVA do relacionamento N:N entre CLIENTE e ENDERECO:
-- um cliente tem vários endereços (casa, trabalho) e um endereço pode
-- ser de vários clientes. O relacionamento ganhou atributos
-- (apelido e principal), por isso virou tabela.
-- ---------------------------------------------------------------------
CREATE TABLE cliente_endereco (
id_cliente INT NOT NULL,
id_endereco INT NOT NULL,
apelido VARCHAR(30) NULL,
principal TINYINT(1) NOT NULL DEFAULT 0,   -- DEFAULT 0: se não informar, assume 0
-- PK COMPOSTA: a combinação (id_cliente, id_endereco) é única, então o
-- mesmo endereço não pode ser vinculado duas vezes ao mesmo cliente.
PRIMARY KEY (id_cliente, id_endereco),
CONSTRAINT fk_ce_cliente
FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)
ON DELETE CASCADE,
-- Para o endereço não usamos CASCADE: um endereço também pode estar em
-- uso por restaurantes ou pedidos, então não pode sumir sem cuidado.
CONSTRAINT fk_ce_endereco
FOREIGN KEY (id_endereco) REFERENCES endereco (id_endereco)
) ENGINE=InnoDB;
  -- ON DELETE CASCADE: se um cliente for excluído, as linhas dele nesta
  -- tabela são excluídas AUTOMATICAMENTE (em cascata). Faz sentido porque
  -- o vínculo não existe sem o cliente.

-- ---------------------------------------------------------------------
-- RESTAURANTE (N:1 com ENDERECO)
-- ---------------------------------------------------------------------
CREATE TABLE restaurante (
id_restaurante INT NOT NULL AUTO_INCREMENT,
nome_fantasia VARCHAR(100) NOT NULL,
cnpj CHAR(14) NOT NULL,  -- CNPJ sem máscara: 14 dígitos
telefone VARCHAR(15)  NULL,
id_endereco INT NOT NULL,
PRIMARY KEY (id_restaurante),
UNIQUE KEY uq_restaurante_cnpj (cnpj),
CONSTRAINT fk_restaurante_endereco
FOREIGN KEY (id_endereco) REFERENCES endereco (id_endereco)
) ENGINE=InnoDB;


-- ---------------------------------------------------------------------
-- CATEGORIA (Pizzas, Bebidas, Sobremesas...)
-- ---------------------------------------------------------------------
CREATE TABLE categoria (
id_categoria INT NOT NULL AUTO_INCREMENT,
```

```
nome VARCHAR(50) NOT NULL,
PRIMARY KEY (id_categoria),
UNIQUE KEY uq_categoria_nome (nome)
) ENGINE=InnoDB;


-- ----------------------------------------------------------------------
-- PRODUTO (N:1 com CATEGORIA e N:1 com RESTAURANTE)
-- ----------------------------------------------------------------------
CREATE TABLE produto (
id_produto INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
descricao VARCHAR(255) NULL,
preco DECIMAL(10,2) NOT NULL,
estoque INT NOT NULL DEFAULT 0,
ativo TINYINT(1) NOT NULL DEFAULT 1,  -- 1 = ativo no cardápio, 0 = inativo
id_categoria INT NOT NULL,
id_restaurante INT NOT NULL,
PRIMARY KEY (id_produto),
CONSTRAINT ck_produto_preco CHECK (preco >= 0),
CONSTRAINT ck_produto_estoque CHECK (estoque >= 0),
CONSTRAINT fk_produto_categoria
FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria),
CONSTRAINT fk_produto_restaurante
FOREIGN KEY (id_restaurante) REFERENCES restaurante (id_restaurante)
) ENGINE=InnoDB;
-- DECIMAL(10,2): número com casas decimais EXATAS: até 10 dígitos no
-- total, sendo 2 depois da vírgula (ex.: 12345678.90). É o tipo certo
-- para dinheiro, pois FLOAT/DOUBLE podem gerar erros de arredondamento.
-- CHECK: regra de validação checada a cada INSERT/UPDATE. Aqui o preço
-- e o estoque não podem ficar negativos. (Exige MySQL 8.0.16 ou superior;
-- em versões mais antigas o CHECK é aceito, mas IGNORADO.)

-- ----------------------------------------------------------------------
-- ENTREGADOR
-- ----------------------------------------------------------------------
CREATE TABLE entregador (
id_entregador INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(100) NOT NULL,
cpf CHAR(11) NOT NULL,
telefone VARCHAR(15) NULL,
tipo_veiculo VARCHAR(20) NOT NULL,       -- Moto, Bicicleta, Carro
placa VARCHAR(8) NULL,           -- NULL para quem entrega de bicicleta
ativo TINYINT(1) NOT NULL DEFAULT 1,
PRIMARY KEY (id_entregador),
UNIQUE KEY uq_entregador_cpf (cpf)
) ENGINE=InnoDB;


-- ----------------------------------------------------------------------
-- PEDIDO
-- Tabela central do sistema. Liga cliente, restaurante, entregador e
-- o endereço de entrega.
-- ----------------------------------------------------------------------
CREATE TABLE pedido (
id_pedido INT NOT NULL AUTO_INCREMENT,
data_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
status ENUM('PENDENTE','PREPARANDO','SAIU_PARA_ENTREGA','ENTREGUE','CANCELADO') NOT NULL DEFAULT
    'PENDENTE',
taxa_entrega DECIMAL(10,2) NOT NULL DEFAULT 0.00,
-- valor_total = soma dos itens + taxa de entrega. É um valor DERIVADO
-- guardado de propósito: o trigger da parte de consultas o atualiza.
valor_total DECIMAL(10,2) NOT NULL DEFAULT 0.00,
id_cliente INT NOT NULL,
id_restaurante INT NOT NULL,
-- id_entregador aceita NULL: o pedido só recebe entregador quando sai
-- para entrega (relacionamento opcional, cardinalidade (0,1)).
id_entregador INT NULL,
-- Endereço onde ESTE pedido será entregue (pode ser diferente do
-- endereço principal do cliente).
id_endereco_entrega INT NOT NULL,
PRIMARY KEY (id_pedido),
CONSTRAINT ck_pedido_taxa CHECK (taxa_entrega >= 0),
CONSTRAINT ck_pedido_total CHECK (valor_total >= 0),
CONSTRAINT fk_pedido_cliente
FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente),
CONSTRAINT fk_pedido_restaurante
FOREIGN KEY (id_restaurante) REFERENCES restaurante (id_restaurante),
CONSTRAINT fk_pedido_entregador
FOREIGN KEY (id_entregador) REFERENCES entregador (id_entregador),
CONSTRAINT fk_pedido_endereco
FOREIGN KEY (id_endereco_entrega) REFERENCES endereco (id_endereco)
) ENGINE=InnoDB;
-- DATETIME: guarda data E hora (AAAA-MM-DD HH:MM:SS).
-- DEFAULT CURRENT_TIMESTAMP: se o INSERT não informar a data/hora, o
```

```sql
-- MySQL grava automaticamente o momento atual do servidor (a função
-- CURRENT_TIMESTAMP é igual a NOW()). Assim o pedido já nasce "datado".
-- ENUM: a coluna só aceita um dos valores da lista. É uma forma de
-- garantir que o status nunca seja escrito errado ("entregue", "ok"...).
-- DEFAULT 'PENDENTE': todo pedido novo começa como pendente.

-- ---------------------------------------------------------------------
-- ITEM_PEDIDO
-- Tabela ASSOCIATIVA do relacionamento N:N entre PEDIDO e PRODUTO:
-- um pedido tem vários produtos e um produto aparece em vários pedidos.
-- preco_unitario guarda o preço NO MOMENTO da compra: se o preço do
-- produto mudar depois, os pedidos antigos continuam com o valor original.
-- ---------------------------------------------------------------------
CREATE TABLE item_pedido (
id_pedido INT NOT NULL,
id_produto INT NOT NULL,
quantidade INT NOT NULL,
preco_unitario DECIMAL(10,2) NOT NULL,
PRIMARY KEY (id_pedido, id_produto), -- PK composta: um produto aparece só uma vez por pedido
CONSTRAINT ck_item_quantidade CHECK (quantidade > 0),
CONSTRAINT ck_item_preco CHECK (preco_unitario >= 0),
-- ON DELETE CASCADE: ao excluir um pedido, os itens dele são excluídos
-- junto (item sem pedido não faz sentido).
CONSTRAINT fk_item_pedido
FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido)
ON DELETE CASCADE,
-- Sem CASCADE em produto: não se pode excluir um produto que já foi vendido.
CONSTRAINT fk_item_produto
FOREIGN KEY (id_produto) REFERENCES produto (id_produto)
) ENGINE=InnoDB;


-- ---------------------------------------------------------------------
-- FORMA_PAGAMENTO (Pix, Cartão de Crédito, Dinheiro...)
-- ---------------------------------------------------------------------
CREATE TABLE forma_pagamento (
id_forma_pagamento INT NOT NULL AUTO_INCREMENT,
descricao VARCHAR(50) NOT NULL,
PRIMARY KEY (id_forma_pagamento),
UNIQUE KEY uq_forma_descricao (descricao)
) ENGINE=InnoDB;


-- ---------------------------------------------------------------------
-- PAGAMENTO (relacionamento 1:1 com PEDIDO)
-- O UNIQUE em id_pedido é o que garante o "1:1": cada pedido tem no
-- máximo um pagamento (sem ele, seria 1:N).
-- ---------------------------------------------------------------------
CREATE TABLE pagamento (
id_pagamento INT NOT NULL AUTO_INCREMENT,
valor DECIMAL(10,2) NOT NULL,
data_pagamento DATE NOT NULL,
status ENUM('PENDENTE','APROVADO','RECUSADO','ESTORNADO') NOT NULL DEFAULT 'PENDENTE',
id_pedido INT NOT NULL,
id_forma_pagamento INT NOT NULL,
PRIMARY KEY (id_pagamento),
UNIQUE KEY uq_pagamento_pedido (id_pedido),
CONSTRAINT ck_pagamento_valor CHECK (valor >= 0),
CONSTRAINT fk_pagamento_pedido
FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido),
CONSTRAINT fk_pagamento_forma
FOREIGN KEY (id_forma_pagamento) REFERENCES forma_pagamento (id_forma_pagamento)
) ENGINE=InnoDB;


-- ---------------------------------------------------------------------
-- AVALIACAO (relacionamento 1:1 OPCIONAL com PEDIDO)
-- Um pedido tem no máximo uma avaliação (UNIQUE em id_pedido), mas
-- pode não ter nenhuma: a linha só existe se o cliente avaliar.
-- ---------------------------------------------------------------------
CREATE TABLE avaliacao (
id_avaliacao INT NOT NULL AUTO_INCREMENT,
-- TINYINT: inteiro pequeno (-128 a 127), suficiente para notas de 1 a 5.
nota TINYINT NOT NULL,
comentario VARCHAR(255) NULL,
data_avaliacao DATE NOT NULL,
id_pedido INT NOT NULL,
PRIMARY KEY (id_avaliacao),
UNIQUE KEY uq_avaliacao_pedido (id_pedido),
-- CHECK ... BETWEEN 1 AND 5: só aceita notas de 1 a 5.
CONSTRAINT ck_avaliacao_nota CHECK (nota BETWEEN 1 AND 5),
CONSTRAINT fk_avaliacao_pedido
FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido)
) ENGINE=InnoDB;
```

Total: 326 linhas.

Apêndice B – Script 02_dados_delivery.sql (completo)

Script DML completo: insere os dados de exemplo (fictícios) em todas as tabelas, respeitando a ordem das chaves estrangeiras.

```sql
-- ================================================================
-- DADOS DE EXEMPLO - Delivery de Restaurante (dados fictícios)

-- Grupo:
-- -> Bernardo Kopp Pinheiro
-- -> Gabriel Valois Rodrigues
-- -> Pedro Henrique Lima dos Santos Mota
-- -> Piêtro Bitencourt Nunes
-- -> Zamorano Fragoso de Sousa

-- DML (Data Manipulation Language): comandos que MANIPULAM os dados
-- (INSERT, UPDATE, DELETE). Aqui usamos só INSERT.
--
-- POR QUE ESTA ORDEM? As tabelas "pai" (as que são apontadas por chave
-- estrangeira) precisam receber os dados ANTES das tabelas "filhas".
-- Exemplo: não dá para inserir um endereço com id_cidade = 1 se a cidade
-- 1 ainda não existe. Por isso a ordem é:
--   cidade -> endereco -> cliente -> cliente_endereco -> restaurante ->
--   categoria -> produto -> entregador -> forma_pagamento -> pedido ->
--   item_pedido -> pagamento -> avaliacao

-- Os ids são informados manualmente nos INSERTs para os relacionamentos
-- (FKs) ficarem fáceis de ler e conferir.
-- ================================================================
USE delivery_restaurante;

-- CIDADE (6): uma em DF e cinco em GO.
INSERT INTO cidade (id_cidade, nome, uf) VALUES
  (1, 'Brasília', 'DF'),
  (2, 'Goiânia', 'GO'),
  (3, 'Anápolis', 'GO'),
  (4, 'Luziânia', 'GO'),
  (5, 'Valparaíso de Goiás', 'GO'),
  (6, 'Águas Lindas de Goiás', 'GO');

-- ENDERECO (21): ids 1 a 4 são dos restaurantes; ids 5 a 21 são de clientes
-- (alguns clientes têm mais de um endereço, ex.: Casa e Trabalho).
INSERT INTO endereco (id_endereco, logradouro, numero, complemento, bairro, cep, id_cidade) VALUES
  (1, 'CLS 308', '12', 'Loja 12', 'Asa Sul', '70747520', 1),
  (2, 'Avenida das Araucárias', '1500', 'Loja 3', 'Águas Claras', '71936250', 1),
  (3, 'QNC 05', '20', NULL, 'Taguatinga', '72110050', 1),
  (4, 'CLN 205', '8', 'Bloco B Loja 8', 'Asa Norte', '70844520', 1),
  (5, 'SQS 114 Bloco C', '302', 'Apto 302', 'Asa Sul', '70377030', 1),
  (6, 'Rua 12 Norte', '45', 'Apto 801', 'Águas Claras', '71909120', 1),
  (7, 'QNM 11', '18', 'Casa', 'Ceilândia', '72215110', 1),
  (8, 'SHCN 705 Bloco A', '204', 'Apto 204', 'Asa Norte', '70730510', 1),
  (9, 'QSC 09', '33', NULL, 'Taguatinga', '72015090', 1),
  (10, 'CRS 504 Bloco A', '103', 'Apto 103', 'Sudoeste', '70350510', 1),
  (11, 'Rua 1', '77', 'Casa', 'Setor Sul', '74093020', 2),
  (12, 'Avenida Brasil', '980', 'Sala 5', 'Centro', '75113050', 3),
  (13, 'Rua das Flores', '15', NULL, 'Jardim Ingá', '72811050', 4),
  (14, 'Quadra 7', '101', 'Casa', 'Parque Esplanada', '72876110', 5),
  (15, 'Rua Bahia', '230', 'Apto 12', 'Setor Oeste', '74125050', 2),
  (16, 'SQN 312 Bloco D', '404', 'Apto 404', 'Asa Norte', '70765040', 1),
  (17, 'QI 17', '09', 'Casa', 'Taguatinga', '72135170', 1),
  (18, 'Rua 8', '60', 'Casa', 'Setor Aeroporto', '74075100', 2),
  (19, 'SQS 205 Bloco F', '101', 'Apto 101', 'Asa Sul', '70235060', 1),
  (20, 'Avenida Castelo Branco', '1200', 'Casa', 'Centro', '72910050', 6),
  (21, 'CSB 04', '16', 'Apto 305', 'Taguatinga', '72015040', 1);

-- CLIENTE (12): os clientes 11 e 12 NÃO têm pedidos, de propósito, para testar
-- consultas com LEFT JOIN / NOT IN (clientes que nunca compraram).
INSERT INTO cliente (id_cliente, nome, cpf, telefone, email, data_cadastro) VALUES
  (1, 'Ana Beatriz Souza', '10000000001', '(61) 99123-0001', 'ana.souza@exemplo.com', '2026-01-15'),
  (2, 'Carlos Eduardo Lima', '10000000002', '(61) 99123-0002', 'carlos.lima@exemplo.com',
      '2026-02-03'),
  (3, 'Mariana Ferreira', '10000000003', '(61) 99123-0003', 'mariana.ferreira@exemplo.com',
      '2026-02-20'),
  (4, 'João Pedro Alves', '10000000004', '(61) 99123-0004', 'joao.alves@exemplo.com', '2026-03-08'),
  (5, 'Fernanda Costa', '10000000005', '(62) 99123-0005', 'fernanda.costa@exemplo.com',
      '2026-03-25'),
  (6, 'Rafael Oliveira', '10000000006', '(62) 99123-0006', 'rafael.oliveira@exemplo.com',
      '2026-04-12'),
```

```sql
  (7, 'Juliana Mendes', '10000000007', '(61) 99123-0007', 'juliana.mendes@exemplo.com',
      '2026-05-02'),
  (8, 'Lucas Martins', '10000000008', '(61) 99123-0008', 'lucas.martins@exemplo.com', '2026-05-30'),
  (9, 'Camila Rocha', '10000000009', '(62) 99123-0009', 'camila.rocha@exemplo.com', '2026-06-18'),
  (10, 'Gabriel Santos', '10000000010', '(61) 99123-0010', 'gabriel.santos@exemplo.com',
      '2026-07-07'),
  (11, 'Patrícia Nogueira', '10000000011', '(61) 99123-0011', 'patricia.nogueira@exemplo.com',
      '2026-09-01'),
  (12, 'Bruno Carvalho', '10000000012', '(61) 99123-0012', 'bruno.carvalho@exemplo.com',
      '2026-10-05');

-- CLIENTE_ENDERECO (17): vínculos N:N entre cliente e endereço.
-- principal = 1 marca o endereço principal de cada cliente.
INSERT INTO cliente_endereco (id_cliente, id_endereco, apelido, principal) VALUES
  (1, 5, 'Casa', 1),
  (1, 8, 'Trabalho', 0),
  (2, 6, 'Casa', 1),
  (3, 7, 'Casa', 1),
  (3, 9, 'Trabalho', 0),
  (4, 10, 'Casa', 1),
  (5, 11, 'Casa', 1),
  (5, 15, 'Trabalho', 0),
  (6, 12, 'Casa', 1),
  (7, 13, 'Casa', 1),
  (8, 14, 'Casa', 1),
  (8, 16, 'Trabalho', 0),
  (9, 17, 'Casa', 1),
  (10, 18, 'Casa', 1),
  (11, 19, 'Casa', 1),
  (11, 21, 'Trabalho', 0),
  (12, 20, 'Casa', 1);

-- RESTAURANTE (4): cada um aponta para um dos endereços 1 a 4.
INSERT INTO restaurante (id_restaurante, nome_fantasia, cnpj, telefone, id_endereco) VALUES
  (1, 'Sabor do Cerrado', '11222333000101', '(61) 3321-0001', 1),
  (2, 'Pizzaria Bella Massa', '11222333000202', '(61) 3321-0002', 2),
  (3, 'Burger Station', '11222333000303', '(61) 3321-0003', 3),
  (4, 'Sushi Hana', '11222333000404', '(61) 3321-0004', 4);

-- CATEGORIA (7): compartilhadas entre os restaurantes (ex.: Bebidas).
INSERT INTO categoria (id_categoria, nome) VALUES
  (1, 'Pratos Principais'),
  (2, 'Pizzas'),
  (3, 'Hambúrgueres'),
  (4, 'Culinária Japonesa'),
  (5, 'Acompanhamentos'),
  (6, 'Bebidas'),
  (7, 'Sobremesas');

-- PRODUTO (24): 6 por restaurante. O produto 3 (Feijoada Individual) está
-- INATIVO e sem estoque (ativo = 0, estoque = 0) para testar filtros.
INSERT INTO produto (id_produto, nome, descricao, preco, estoque, ativo, id_categoria,
    id_restaurante) VALUES
  (1, 'Baião de Dois com Carne de Sol', 'Baião de dois cremoso com carne de sol desfiada', 34.90,
      60, 1, 1, 1),
  (2, 'Galinhada com Pequi', 'Arroz com frango caipira e pequi', 32.00, 45, 1, 1, 1),
  (3, 'Feijoada Individual', 'Feijoada completa para uma pessoa', 38.50, 0, 0, 1, 1),
  (4, 'Pamonha Salgada', 'Pamonha de milho com queijo', 12.00, 80, 1, 5, 1),
  (5, 'Suco de Cajá', 'Suco natural de cajá 500 ml', 9.50, 70, 1, 6, 1),
  (6, 'Pudim de Leite', 'Pudim de leite condensado', 11.00, 40, 1, 7, 1),
  (7, 'Pizza Margherita', 'Molho, muçarela, tomate e manjericão', 49.90, 50, 1, 2, 2),
  (8, 'Pizza Calabresa', 'Calabresa fatiada com cebola', 54.90, 50, 1, 2, 2),
  (9, 'Pizza Quatro Queijos', 'Muçarela, provolone, gorgonzola e parmesão', 59.90, 40, 1, 2, 2),
  (10, 'Pizza Frango com Catupiry', 'Frango desfiado com catupiry', 57.90, 35, 1, 2, 2),
  (11, 'Refrigerante 2L', 'Refrigerante sabor cola 2 litros', 14.00, 100, 1, 6, 2),
  (12, 'Petit Gâteau', 'Bolo quente de chocolate com sorvete', 18.90, 30, 1, 7, 2),
  (13, 'X-Burger', 'Hambúrguer, queijo e molho especial', 26.90, 90, 1, 3, 3),
  (14, 'X-Bacon', 'Hambúrguer, queijo e bacon crocante', 31.90, 85, 1, 3, 3),
  (15, 'X-Salada', 'Hambúrguer, queijo, alface e tomate', 24.90, 75, 1, 3, 3),
  (16, 'Batata Frita Grande', 'Porção grande de batata frita', 19.90, 120, 1, 5, 3),
  (17, 'Milkshake de Chocolate', 'Milkshake 400 ml', 17.90, 60, 1, 7, 3),
  (18, 'Refrigerante Lata', 'Refrigerante lata 350 ml', 6.50, 200, 1, 6, 3),
  (19, 'Combo 20 Peças', 'Sushis e sashimis variados', 62.00, 30, 1, 4, 4),
  (20, 'Temaki de Salmão', 'Temaki de salmão com cream cheese', 28.00, 45, 1, 4, 4),
  (21, 'Hot Roll 8 Unidades', 'Hot roll de salmão empanado', 29.90, 40, 1, 4, 4),
  (22, 'Yakisoba de Frango', 'Macarrão oriental com frango e legumes', 36.00, 35, 1, 1, 4),
  (23, 'Chá Gelado', 'Chá gelado de pêssego 500 ml', 8.00, 90, 1, 6, 4),
  (24, 'Mochi Sorvete', 'Mochi recheado com sorvete', 15.00, 25, 1, 7, 4);

-- ENTREGADOR (6): o 4 (Leandro) está inativo e o 6 (Felipe) nunca fez entrega,
-- para testar consultas com LEFT JOIN e COUNT. Quem usa bicicleta não tem placa (NULL).
INSERT INTO entregador (id_entregador, nome, cpf, telefone, tipo_veiculo, placa, ativo) VALUES
  (1, 'Marcos Vinícius Pereira', '20000000001', '(61) 98111-0001', 'Moto', 'PBX1A23', 1),
  (2, 'Diego Ramos', '20000000002', '(61) 98111-0002', 'Moto', 'OVD4F56', 1),
  (3, 'Thiago Barbosa', '20000000003', '(61) 98111-0003', 'Bicicleta', NULL, 1),
```

```sql
  (4, 'Leandro Cunha', '20000000004', '(61) 98111-0004', 'Moto', 'JKL7H89', 0),
  (5, 'Rodrigo Teixeira', '20000000005', '(61) 98111-0005', 'Carro', 'NGB2C34', 1),
  (6, 'Felipe Araújo', '20000000006', '(61) 98111-0006', 'Bicicleta', NULL, 1);

-- FORMA_PAGAMENTO (5).
INSERT INTO forma_pagamento (id_forma_pagamento, descricao) VALUES
  (1, 'Cartão de Crédito'),
  (2, 'Cartão de Débito'),
  (3, 'Pix'),
  (4, 'Dinheiro'),
  (5, 'Vale-Refeição');

-- PEDIDO (33): de agosto a novembro de 2026, com todos os status do ENUM.
-- id_entregador = NULL nos pedidos que ainda não saíram para entrega e nos cancelados.
-- valor_total = soma dos itens + taxa_entrega (já calculado).
INSERT INTO pedido (id_pedido, data_hora, status, taxa_entrega, valor_total, id_cliente,
    id_restaurante, id_entregador, id_endereco_entrega) VALUES
  (1, '2026-08-01 22:04:00', 'ENTREGUE', 6.00, 39.00, 9, 1, 2, 17),
  (2, '2026-08-03 13:11:00', 'ENTREGUE', 8.00, 351.90, 2, 4, 2, 6),
  (3, '2026-08-15 08:26:00', 'ENTREGUE', 8.00, 95.00, 9, 4, 4, 17),
  (4, '2026-08-16 03:41:00', 'CANCELADO', 8.00, 195.80, 10, 4, NULL, 18),
  (5, '2026-08-17 18:45:00', 'ENTREGUE', 6.00, 205.30, 7, 1, 3, 13),
  (6, '2026-08-19 07:23:00', 'ENTREGUE', 7.50, 225.10, 8, 2, 5, 16),
  (7, '2026-08-20 03:25:00', 'ENTREGUE', 6.00, 17.00, 2, 1, 4, 6),
  (8, '2026-08-21 14:48:00', 'ENTREGUE', 7.50, 251.00, 4, 2, 4, 10),
  (9, '2026-08-23 03:26:00', 'ENTREGUE', 8.00, 60.00, 10, 4, 4, 18),
  (10, '2026-09-08 17:29:00', 'ENTREGUE', 5.00, 72.70, 9, 3, 5, 17),
  (11, '2026-09-11 02:32:00', 'ENTREGUE', 8.00, 168.00, 7, 4, 4, 13),
  (12, '2026-09-14 03:17:00', 'ENTREGUE', 6.00, 75.80, 2, 1, 3, 6),
  (13, '2026-09-16 05:52:00', 'ENTREGUE', 5.00, 116.50, 8, 3, 5, 16),
  (14, '2026-09-22 17:58:00', 'ENTREGUE', 5.00, 174.30, 3, 3, 3, 9),
  (15, '2026-09-27 15:54:00', 'ENTREGUE', 6.00, 120.20, 3, 1, 5, 7),
  (16, '2026-09-28 14:50:00', 'ENTREGUE', 5.00, 94.60, 9, 3, 2, 17),
  (17, '2026-10-02 10:12:00', 'ENTREGUE', 7.50, 177.20, 7, 2, 3, 13),
  (18, '2026-10-03 17:56:00', 'ENTREGUE', 8.00, 83.80, 8, 4, 5, 16),
  (19, '2026-10-05 09:10:00', 'ENTREGUE', 5.00, 78.30, 2, 3, 2, 6),
  (20, '2026-10-08 23:16:00', 'ENTREGUE', 8.00, 140.80, 6, 4, 1, 12),
  (21, '2026-10-12 18:28:00', 'ENTREGUE', 5.00, 81.70, 5, 3, 3, 15),
  (22, '2026-10-17 00:41:00', 'ENTREGUE', 7.50, 95.20, 3, 2, 3, 7),
  (23, '2026-10-20 18:10:00', 'ENTREGUE', 7.50, 86.30, 5, 2, 3, 15),
  (24, '2026-10-22 07:34:00', 'ENTREGUE', 5.00, 31.90, 7, 3, 1, 13),
  (25, '2026-10-29 17:05:00', 'ENTREGUE', 5.00, 128.50, 1, 3, 1, 5),
  (26, '2026-10-29 19:47:00', 'CANCELADO', 8.00, 23.00, 4, 4, NULL, 10),
  (27, '2026-10-31 22:31:00', 'ENTREGUE', 5.00, 98.10, 9, 3, 3, 17),
  (28, '2026-11-02 15:11:00', 'CANCELADO', 7.50, 200.10, 5, 2, NULL, 15),
  (29, '2026-11-02 22:53:00', 'ENTREGUE', 7.50, 117.30, 2, 2, 1, 6),
  (30, '2026-11-10 19:40:00', 'SAIU_PARA_ENTREGA', 5.00, 94.60, 6, 3, 5, 12),
  (31, '2026-11-10 20:05:00', 'PREPARANDO', 8.00, 60.00, 4, 4, NULL, 10),
  (32, '2026-11-11 11:30:00', 'PENDENTE', 8.00, 98.00, 9, 4, NULL, 17),
  (33, '2026-11-11 12:10:00', 'PENDENTE', 8.00, 155.90, 9, 4, NULL, 17);

-- ITEM_PEDIDO (78): itens de cada pedido. preco_unitario é o preço no momento
-- da compra. Todos os itens de um pedido são do restaurante daquele pedido.
INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES
  (1, 6, 3, 11.00),
  (2, 20, 2, 28.00),
  (2, 19, 3, 62.00),
  (2, 22, 2, 36.00),
  (2, 21, 1, 29.90),
  (3, 22, 1, 36.00),
  (3, 24, 1, 15.00),
  (3, 20, 1, 28.00),
  (3, 23, 1, 8.00),
  (4, 20, 2, 28.00),
  (4, 21, 2, 29.90),
  (4, 22, 2, 36.00),
  (5, 1, 2, 34.90),
  (5, 5, 1, 9.50),
  (5, 4, 2, 12.00),
  (5, 2, 3, 32.00),
  (6, 7, 1, 49.90),
  (6, 8, 2, 54.90),
  (6, 10, 1, 57.90),
  (7, 6, 1, 11.00),
  (8, 12, 1, 18.90),
  (8, 9, 1, 59.90),
  (8, 8, 3, 54.90),
  (9, 22, 1, 36.00),
  (9, 23, 2, 8.00),
  (10, 14, 1, 31.90),
  (10, 17, 2, 17.90),
  (11, 22, 1, 36.00),
  (11, 19, 2, 62.00),
  (12, 1, 2, 34.90),
  (13, 13, 2, 26.90),
```

```sql
  (13, 16, 2, 19.90),
  (13, 17, 1, 17.90),
  (14, 17, 2, 17.90),
  (14, 14, 1, 31.90),
  (14, 13, 1, 26.90),
  (14, 15, 3, 24.90),
  (15, 5, 1, 9.50),
  (15, 1, 3, 34.90),
  (16, 16, 2, 19.90),
  (16, 15, 2, 24.90),
  (17, 9, 1, 59.90),
  (17, 8, 2, 54.90),
  (18, 21, 2, 29.90),
  (18, 23, 2, 8.00),
  (19, 13, 2, 26.90),
  (19, 18, 3, 6.50),
  (20, 20, 1, 28.00),
  (20, 21, 2, 29.90),
  (20, 24, 3, 15.00),
  (21, 13, 1, 26.90),
  (21, 15, 2, 24.90),
  (22, 12, 2, 18.90),
  (22, 7, 1, 49.90),
  (23, 9, 1, 59.90),
  (23, 12, 1, 18.90),
  (24, 13, 1, 26.90),
  (25, 15, 2, 24.90),
  (25, 16, 1, 19.90),
  (25, 13, 2, 26.90),
  (26, 24, 1, 15.00),
  (27, 13, 1, 26.90),
  (27, 16, 3, 19.90),
  (27, 18, 1, 6.50),
  (28, 9, 1, 59.90),
  (28, 11, 1, 14.00),
  (28, 7, 2, 49.90),
  (28, 12, 1, 18.90),
  (29, 8, 2, 54.90),
  (30, 15, 2, 24.90),
  (30, 16, 2, 19.90),
  (31, 20, 1, 28.00),
  (31, 23, 3, 8.00),
  (32, 20, 1, 28.00),
  (32, 19, 1, 62.00),
  (33, 21, 1, 29.90),
  (33, 20, 2, 28.00),
  (33, 19, 1, 62.00);

-- PAGAMENTO (33): um por pedido (relacionamento 1:1). O valor é igual ao
-- valor_total do pedido; o status varia conforme a situação do pedido.
INSERT INTO pagamento (id_pagamento, valor, data_pagamento, status, id_pedido, id_forma_pagamento)
    VALUES
  (1, 39.00, '2026-08-01', 'APROVADO', 1, 3),
  (2, 351.90, '2026-08-03', 'APROVADO', 2, 5),
  (3, 95.00, '2026-08-15', 'APROVADO', 3, 3),
  (4, 195.80, '2026-08-16', 'RECUSADO', 4, 2),
  (5, 205.30, '2026-08-17', 'APROVADO', 5, 1),
  (6, 225.10, '2026-08-19', 'APROVADO', 6, 4),
  (7, 17.00, '2026-08-20', 'APROVADO', 7, 3),
  (8, 251.00, '2026-08-21', 'APROVADO', 8, 2),
  (9, 60.00, '2026-08-23', 'APROVADO', 9, 1),
  (10, 72.70, '2026-09-08', 'APROVADO', 10, 1),
  (11, 168.00, '2026-09-11', 'APROVADO', 11, 3),
  (12, 75.80, '2026-09-14', 'APROVADO', 12, 1),
  (13, 116.50, '2026-09-16', 'APROVADO', 13, 3),
  (14, 174.30, '2026-09-22', 'APROVADO', 14, 5),
  (15, 120.20, '2026-09-27', 'APROVADO', 15, 3),
  (16, 94.60, '2026-09-28', 'APROVADO', 16, 2),
  (17, 177.20, '2026-10-02', 'APROVADO', 17, 4),
  (18, 83.80, '2026-10-03', 'APROVADO', 18, 3),
  (19, 78.30, '2026-10-05', 'APROVADO', 19, 2),
  (20, 140.80, '2026-10-08', 'APROVADO', 20, 5),
  (21, 81.70, '2026-10-12', 'APROVADO', 21, 2),
  (22, 95.20, '2026-10-17', 'APROVADO', 22, 3),
  (23, 86.30, '2026-10-20', 'APROVADO', 23, 3),
  (24, 31.90, '2026-10-22', 'APROVADO', 24, 4),
  (25, 128.50, '2026-10-29', 'APROVADO', 25, 5),
  (26, 23.00, '2026-10-29', 'RECUSADO', 26, 3),
  (27, 98.10, '2026-10-31', 'APROVADO', 27, 1),
  (28, 200.10, '2026-11-02', 'ESTORNADO', 28, 3),
  (29, 117.30, '2026-11-02', 'APROVADO', 29, 3),
  (30, 94.60, '2026-11-10', 'APROVADO', 30, 1),
  (31, 60.00, '2026-11-10', 'APROVADO', 31, 2),
  (32, 98.00, '2026-11-11', 'PENDENTE', 32, 5),
  (33, 155.90, '2026-11-11', 'PENDENTE', 33, 3);
```

```sql

-- AVALIACAO (19): só alguns pedidos ENTREGUES foram avaliados (relacionamento
-- opcional), com notas de 1 a 5.
INSERT INTO avaliacao (id_avaliacao, nota, comentario, data_avaliacao, id_pedido) VALUES
  (1, 5, 'Chegou rápido e muito saboroso', '2026-08-01', 1),
  (2, 4, 'Comida boa e bem embalada', '2026-08-03', 2),
  (3, 2, 'Demorou demais', '2026-08-18', 5),
  (4, 1, 'Pedido veio errado', '2026-08-19', 6),
  (5, 5, 'Perfeito, recomendo', '2026-08-22', 8),
  (6, 5, 'Tudo excelente', '2026-08-23', 9),
  (7, 5, 'Tudo excelente', '2026-09-09', 10),
  (8, 1, 'Pedido veio errado', '2026-09-11', 11),
  (9, 1, 'Pedido veio errado', '2026-09-16', 13),
  (10, 5, 'Chegou rápido e muito saboroso', '2026-09-27', 15),
  (11, 5, 'Tudo excelente', '2026-09-28', 16),
  (12, 5, 'Tudo excelente', '2026-10-03', 18),
  (13, 5, 'Chegou rápido e muito saboroso', '2026-10-06', 19),
  (14, 2, 'Demorou demais', '2026-10-10', 20),
  (15, 5, 'Chegou rápido e muito saboroso', '2026-10-13', 21),
  (16, 4, 'Gostei bastante', '2026-10-17', 22),
  (17, 4, 'Gostei bastante', '2026-10-23', 24),
  (18, 3, 'Razoável, esperava mais', '2026-10-31', 25),
  (19, 4, 'Muito bom, só demorou um pouco', '2026-11-03', 29);
```

Total: 339 linhas.

Apêndice C – Script 03_consultas_delivery.sql (completo)

Script com as 26 consultas (6 básicas, 15 intermediárias e 5 avançadas), a criação dos triggers e procedures e os testes executados.

```sql
-- ========================================================================
-- PROJETO FINAL - MODELAGEM DE BANCO DE DADOS
-- Tema: Delivery de Restaurante
-- CONSULTAS SQL (6 básicas, 15 intermediárias, 5 avançadas)
-- DQL (Data Query Language): comandos de CONSULTA (SELECT).
-- Cada consulta tem: o que ela responde, o conceito de SQL usado e o
-- comando. No Workbench, selecione UMA consulta por vez e execute
-- (Ctrl+Shift+Enter) para ver o resultado de cada uma no print.
-- ========================================================================
USE delivery_restaurante;


-- ========================================================================
--               CONSULTAS BÁSICAS  (SELECT, FROM)
-- ========================================================================

-- B1. Listar todos os dados de todos os clientes.
-- Conceito: SELECT * traz todas as colunas da tabela.
SELECT * FROM cliente;

-- B2. Listar apenas nome, telefone e e-mail dos clientes.
-- Conceito: escolher colunas específicas em vez de usar *.
SELECT nome, telefone, email
FROM cliente;

-- B3. Mostrar o cardápio completo (nome e preço dos produtos).
-- Conceito: apelido de coluna com AS (o título do resultado muda, a tabela não).
SELECT nome AS produto, preco AS preco_reais
FROM produto;

-- B4. Listar os restaurantes cadastrados.
SELECT nome_fantasia, cnpj, telefone
FROM restaurante;

-- B5. Listar os entregadores e seus veículos.
SELECT nome AS entregador, tipo_veiculo, placa
FROM entregador;

-- B6. Listar os pedidos com data, status e valor total.
SELECT id_pedido, data_hora, status, valor_total
FROM pedido;


-- ========================================================================
--                     CONSULTAS INTERMEDIÁRIAS
-- ========================================================================

-- I1. Quais tipos de veículo os entregadores usam (sem repetição)?
-- Conceito: DISTINCT remove linhas repetidas do resultado.
SELECT DISTINCT tipo_veiculo
FROM entregador;

-- I2. Mostrar o valor total em estoque de cada produto ativo e um preço
--     promocional com 10% de desconto.
-- Conceito: colunas calculadas (*, -, /) e ROUND para arredondar.
SELECT nome, preco, estoque,
preco * estoque AS valor_em_estoque,
ROUND(preco * 0.90,2) AS preco_promocional
FROM produto
WHERE ativo = 1;

-- I3. Produtos ativos caros (>= 50) OU com estoque baixo (< 30), exceto bebidas.
-- Conceito: WHERE com operadores de comparação (>=, <, =) e lógicos
-- (AND, OR, NOT). Os parênteses definem a ordem de avaliação.
SELECT nome, preco, estoque, id_categoria
FROM produto
WHERE ativo = 1
AND (preco >= 50 OR estoque < 30)
AND NOT (id_categoria = 6);

-- I4. Buscar produtos de pizza ou lanches "X-".
-- Conceito: LIKE com curinga % (qualquer sequência de caracteres).
SELECT nome, preco
```

```sql
FROM produto
WHERE nome LIKE '%Pizza%'
   OR nome LIKE 'X-%';

-- I5. Pedidos feitos em outubro de 2026.
-- Conceito: BETWEEN (intervalo com os extremos incluídos).
SELECT id_pedido, data_hora, status, valor_total
FROM pedido
WHERE data_hora BETWEEN '2026-10-01 00:00:00' AND '2026-10-31 23:59:59'
ORDER BY data_hora;

-- I6. Pedidos que ainda estão em andamento.
-- Conceito: IN (compara com uma lista de valores; evita vários OR).
SELECT id_pedido, data_hora, status, valor_total
FROM pedido
WHERE status IN ('PENDENTE','PREPARANDO','SAIU_PARA_ENTREGA');

-- I7. Resumo financeiro dos pedidos entregues: quantidade, faturamento,
--     ticket médio, maior e menor pedido.
-- Conceito: funções de agregação COUNT, SUM, AVG, MAX e MIN.
SELECT COUNT(*) AS qtd_pedidos,
SUM(valor_total) AS faturamento,
ROUND(AVG(valor_total),2) AS ticket_medio,
MAX(valor_total) AS maior_pedido,
MIN(valor_total) AS menor_pedido
FROM pedido
WHERE status = 'ENTREGUE';

-- I8. Pedidos com cliente, restaurante, bairro e cidade de entrega.
-- Conceito: INNER JOIN liga tabelas pela chave estrangeira (FK = PK).
-- Só aparecem linhas que têm correspondência nas tabelas ligadas.
SELECT p.id_pedido, c.nome AS cliente, r.nome_fantasia AS restaurante, en.bairro AS bairro_entrega,
    ci.nome AS cidade_entrega, p.valor_total
FROM pedido p
INNER JOIN cliente c ON c.id_cliente = p.id_cliente
INNER JOIN restaurante r ON r.id_restaurante = p.id_restaurante
INNER JOIN endereco en ON en.id_endereco = p.id_endereco_entrega
INNER JOIN cidade ci ON ci.id_cidade = en.id_cidade
ORDER BY p.id_pedido;

-- I9. Os 5 produtos mais vendidos (em unidades), desconsiderando cancelados.
-- Conceito: GROUP BY agrupa linhas; SUM soma por grupo; ORDER BY ... DESC
-- ordena do maior para o menor; LIMIT corta o resultado.
SELECT pr.nome AS produto,
SUM(i.quantidade) AS total_vendido
FROM item_pedido i
INNER JOIN produto pr ON pr.id_produto = i.id_produto
INNER JOIN pedido p ON p.id_pedido = i.id_pedido
WHERE p.status <> 'CANCELADO'
GROUP BY pr.id_produto, pr.nome
ORDER BY total_vendido DESC, produto
LIMIT 5;

-- I10. Restaurantes com faturamento (pedidos entregues) acima de R$ 500.
-- Conceito: HAVING filtra DEPOIS do agrupamento (WHERE filtra antes e
-- não pode usar SUM/COUNT/AVG).
SELECT r.nome_fantasia AS restaurante,
COUNT(p.id_pedido) AS qtd_pedidos,
SUM(p.valor_total) AS faturamento
FROM restaurante r
INNER JOIN pedido p ON p.id_restaurante = r.id_restaurante
WHERE p.status = 'ENTREGUE'
GROUP BY r.id_restaurante, r.nome_fantasia
HAVING SUM(p.valor_total) > 500
ORDER BY faturamento DESC;

-- I11. Nota média e quantidade de avaliações de cada restaurante.
-- Conceito: AVG com JOIN de 3 tabelas (avaliacao -> pedido -> restaurante).
SELECT r.nome_fantasia AS restaurante,
COUNT(a.id_avaliacao) AS qtd_avaliacoes,
ROUND(AVG(a.nota),2) AS nota_media
FROM avaliacao a
INNER JOIN pedido p ON p.id_pedido = a.id_pedido
INNER JOIN restaurante r ON r.id_restaurante = p.id_restaurante
GROUP BY r.id_restaurante, r.nome_fantasia
ORDER BY nota_media DESC;

-- I12. Produtos ativos com preço acima da média de preços.
-- Conceito: SUBCONSULTA escalar (retorna um único valor) dentro do WHERE.
SELECT nome, preco
FROM produto
WHERE ativo = 1 AND preco > (SELECT AVG(preco) FROM produto WHERE ativo = 1)
ORDER BY preco DESC;

-- I13. Clientes que nunca fizeram um pedido.
```

```sql
-- Conceito: SUBCONSULTA com NOT IN (a lista vem de outra consulta).
SELECT id_cliente, nome, data_cadastro
FROM cliente
WHERE id_cliente NOT IN (SELECT id_cliente FROM pedido);

-- I14. Quantidade de entregas concluídas por entregador, incluindo quem
--      nunca entregou.
-- Conceito: LEFT JOIN mantém TODAS as linhas da tabela da esquerda
-- (entregador), mesmo sem correspondência. COUNT(p.id_pedido) ignora
-- NULL, então quem nunca entregou aparece com 0.
SELECT e.nome AS entregador, e.tipo_veiculo, e.ativo,
COUNT(p.id_pedido) AS entregas
FROM entregador e
LEFT JOIN pedido p
ON p.id_entregador = e.id_entregador
AND p.status = 'ENTREGUE'
GROUP BY e.id_entregador, e.nome, e.tipo_veiculo, e.ativo
ORDER BY entregas DESC;

-- I15. Agenda de contatos: nome e telefone de clientes e entregadores.
-- Conceito: UNION junta os resultados de duas consultas (mesmo número
-- de colunas e tipos compatíveis) e remove duplicados.
SELECT nome, telefone, 'Cliente' AS tipo FROM cliente
UNION
SELECT nome, telefone, 'Entregador' AS tipo FROM entregador
ORDER BY nome;


-- ======================================================================
--            CONSULTAS AVANÇADAS  (TRIGGER e STORED PROCEDURE)
--
-- TRIGGER (gatilho): código que o MySQL executa SOZINHO quando acontece
--   um INSERT, UPDATE ou DELETE em uma tabela (BEFORE = antes, AFTER = depois).
--   Dentro do trigger, NEW.coluna é o valor novo e OLD.coluna é o valor antigo.
-- STORED PROCEDURE (procedimento armazenado): bloco de código salvo no
--   banco, executado quando chamado com CALL, e que pode receber parâmetros.
--
-- DELIMITER: o MySQL usa ";" para terminar um comando, mas dentro de
-- triggers/procedures existem vários ";". Por isso trocamos o delimitador
-- para $$ durante a criação e depois voltamos para ";".
-- ======================================================================

DELIMITER $$

-- ----------------------------------------------------------------------
-- A1. TRIGGER BEFORE INSERT em item_pedido: validar o item ANTES de gravar.
-- Regras: o produto precisa estar ativo, ser do restaurante do pedido e
-- ter estoque suficiente. Se alguma regra falhar, SIGNAL cancela o INSERT
-- e mostra uma mensagem de erro (SQLSTATE '45000' = erro definido pelo usuário).
-- ----------------------------------------------------------------------
CREATE TRIGGER trg_item_valida_antes_inserir
BEFORE INSERT ON item_pedido
FOR EACH ROW
BEGIN
  DECLARE v_estoque INT;
  DECLARE v_ativo TINYINT;
  DECLARE v_rest_produto INT;
  DECLARE v_rest_pedido INT;

  SELECT estoque, ativo, id_restaurante
    INTO v_estoque, v_ativo, v_rest_produto
  FROM produto
  WHERE id_produto = NEW.id_produto;

  SELECT id_restaurante INTO v_rest_pedido
  FROM pedido
  WHERE id_pedido = NEW.id_pedido;

  IF v_ativo = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Produto inativo: nao pode ser vendido.';
  END IF;

  IF v_rest_produto <> v_rest_pedido THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'O produto nao pertence ao restaurante do pedido.';
  END IF;

  IF v_estoque < NEW.quantidade THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Estoque insuficiente para este produto.';
  END IF;
END$$

-- ----------------------------------------------------------------------
-- A2. TRIGGER AFTER INSERT em item_pedido: depois de gravar o item,
-- (1) dá baixa no estoque do produto e (2) recalcula o valor_total do pedido
-- (soma dos itens + taxa de entrega).
```

```sql
-- ---------------------------------------------------------------------
CREATE TRIGGER trg_item_apos_inserir
AFTER INSERT ON item_pedido
FOR EACH ROW
BEGIN
  UPDATE produto
     SET estoque = estoque - NEW.quantidade
   WHERE id_produto = NEW.id_produto;

  UPDATE pedido
     SET valor_total = taxa_entrega +
         (SELECT SUM(quantidade * preco_unitario)
            FROM item_pedido
           WHERE id_pedido = NEW.id_pedido)
   WHERE id_pedido = NEW.id_pedido;
END$$

-- ---------------------------------------------------------------------
-- A3. TRIGGER AFTER UPDATE em pedido: quando o status muda para CANCELADO,
-- devolve ao estoque as quantidades dos itens do pedido.
-- OLD.status <> NEW.status evita devolver duas vezes se o pedido já estava cancelado.
-- ---------------------------------------------------------------------
CREATE TRIGGER trg_pedido_cancelado_devolve_estoque
AFTER UPDATE ON pedido
FOR EACH ROW
BEGIN
  IF NEW.status = 'CANCELADO' AND OLD.status <> 'CANCELADO' THEN
    UPDATE produto pr
    INNER JOIN item_pedido i ON i.id_produto = pr.id_produto
       SET pr.estoque = pr.estoque + i.quantidade
     WHERE i.id_pedido = NEW.id_pedido;
  END IF;
END$$

-- ---------------------------------------------------------------------
-- A4. STORED PROCEDURE sp_adicionar_item: adiciona um produto a um pedido.
-- Busca o preço atual do produto e insere o item usando esse preço.
-- Os triggers A1 e A2 validam e atualizam estoque e total automaticamente.
-- Parâmetros IN: valores recebidos na chamada (CALL).
-- ---------------------------------------------------------------------
CREATE PROCEDURE sp_adicionar_item(
  IN p_id_pedido  INT,
  IN p_id_produto INT,
  IN p_quantidade INT
)
BEGIN
  DECLARE v_preco DECIMAL(10,2);

  SELECT preco INTO v_preco
  FROM produto
  WHERE id_produto = p_id_produto;

  IF v_preco IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Produto nao encontrado.';
  END IF;

  INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario)
  VALUES (p_id_pedido, p_id_produto, p_quantidade, v_preco);
END$$

-- ---------------------------------------------------------------------
-- A5. STORED PROCEDURE sp_faturamento_restaurante: relatório de um
-- restaurante em um período (só pedidos ENTREGUES).
-- Retorna: quantidade de pedidos, faturamento e ticket médio.
-- COALESCE troca NULL por 0 quando não há pedidos no período.
-- ---------------------------------------------------------------------
CREATE PROCEDURE sp_faturamento_restaurante(
  IN p_id_restaurante INT,
  IN p_data_inicio    DATE,
  IN p_data_fim       DATE
)
BEGIN
  SELECT r.nome_fantasia AS restaurante,
         COUNT(p.id_pedido)                          AS qtd_pedidos,
         COALESCE(SUM(p.valor_total),0)              AS faturamento,
         COALESCE(ROUND(AVG(p.valor_total),2),0)     AS ticket_medio
  FROM restaurante r
  LEFT JOIN pedido p
         ON p.id_restaurante = r.id_restaurante
        AND p.status = 'ENTREGUE'
        AND DATE(p.data_hora) BETWEEN p_data_inicio AND p_data_fim
  WHERE r.id_restaurante = p_id_restaurante
  GROUP BY r.id_restaurante, r.nome_fantasia;
END$$

```

```
DELIMITER ;


-- ======================================================================
-- TESTES DAS CONSULTAS AVANÇADAS
-- Cada teste fica dentro de uma TRANSAÇÃO que termina com ROLLBACK:
-- os comandos rodam e mostram o resultado, mas os dados voltam ao que
-- eram antes (nada é gravado de verdade).
-- ----------------------------------------------------------------------

-- Teste A5 (procedure de relatório): faturamento do Sushi Hana (id 4) em outubro.
CALL sp_faturamento_restaurante(4, '2026-10-01', '2026-10-31');

-- Teste A4 + A2 (procedure de item + trigger de estoque e total):
-- adiciona 2 unidades do Yakisoba (produto 22) ao pedido 33 (Sushi Hana).
START TRANSACTION;
  SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- antes
  SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- antes
  CALL sp_adicionar_item(33, 22, 2);
  SELECT id_produto, nome, estoque FROM produto WHERE id_produto = 22;      -- depois: estoque -2
  SELECT id_pedido, taxa_entrega, valor_total FROM pedido WHERE id_pedido = 33;  -- depois: total +
      72,00
ROLLBACK;

-- Teste A3 (cancelamento devolve estoque): cancela o pedido 33 e mostra o estoque dos itens.
START TRANSACTION;
  SELECT i.id_produto, pr.nome, pr.estoque AS estoque_antes
  FROM item_pedido i INNER JOIN produto pr ON pr.id_produto = i.id_produto
  WHERE i.id_pedido = 33;
  UPDATE pedido SET status = 'CANCELADO' WHERE id_pedido = 33;
  SELECT i.id_produto, pr.nome, pr.estoque AS estoque_depois, i.quantidade AS devolvido
  FROM item_pedido i INNER JOIN produto pr ON pr.id_produto = i.id_produto
  WHERE i.id_pedido = 33;
ROLLBACK;

-- Teste A1 (validações): descomente UMA linha por vez para ver cada erro.
-- Obs.: no Workbench, um erro interrompe a execução do script; por isso
-- rode estas linhas isoladas.
-- CALL sp_adicionar_item(33, 3, 1);    -- erro: produto inativo (Feijoada)
-- CALL sp_adicionar_item(33, 7, 1);    -- erro: produto de outro restaurante (pizza em pedido de
    sushi)
-- CALL sp_adicionar_item(33, 19, 999); -- erro: estoque insuficiente
-- CALL sp_adicionar_item(33, 99, 1);   -- erro: produto nao encontrado
```

Total: 368 linhas.